Beyond Standard LLMs

Linear Attention Hybrids, Text Diffusion, Code World Models, and Small Recursive Transforr

SEBASTIAN RASCHKA, PHD
NOV 04, 2025

384 28 37 Share

From DeepSeek R1 to MiniMax-M2, the largest and most capable open-weight LLMs t remain autoregressive decoder-style transformers, which are built on flavors of the orig multi-head attention mechanism.

However, we have also seen alternatives to standard LLMs popping up in recent years, text diffusion models to the most recent linear attention hybrid architectures. Some of t are geared towards better efficiency, and others, like code world models, aim to improv modeling performance.

After I shared my Big LLM Architecture Comparison a few months ago, which focused main transformer-based LLMs, I received a lot of questions with respect to what I think alternative approaches. (I also recently gave a short talk about that at the PyTorch Conference 2025, where I also promised attendees to follow up with a write-up of thes alternative approaches). So here it is!

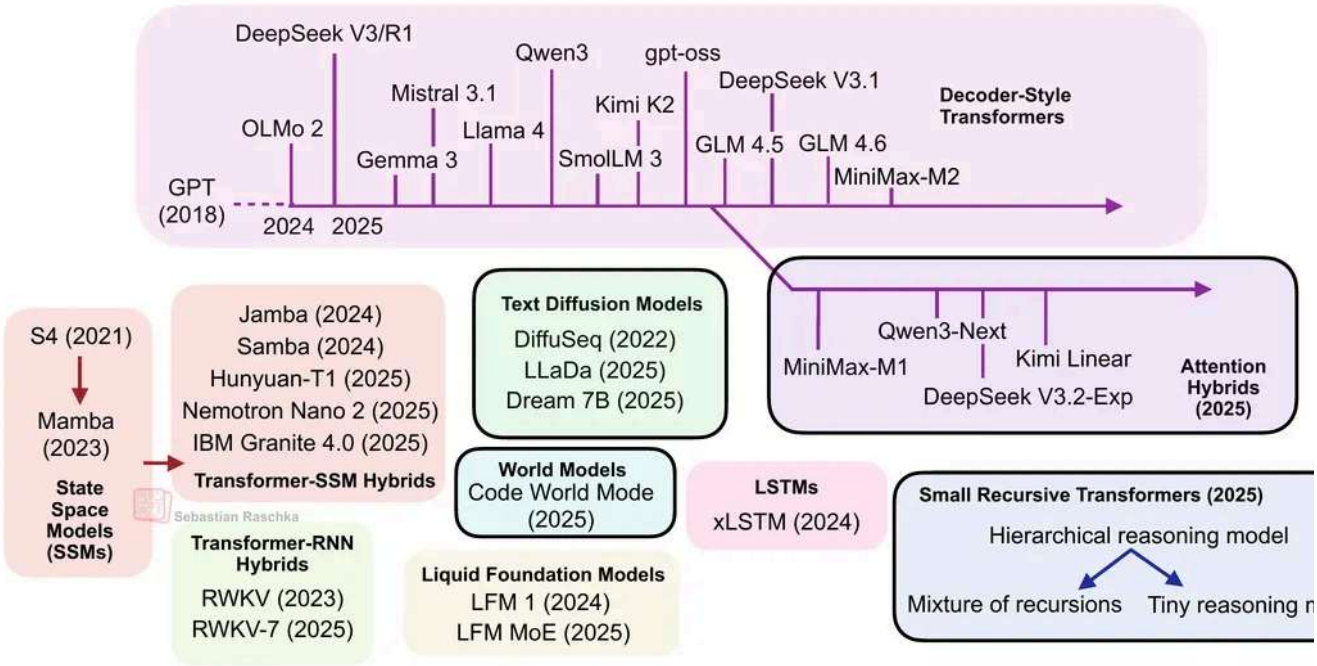


Figure 1: Overview of the LLM landscape. This article covers those architectures surrounded by the black frames. The decoder-style transformers are covered in my "The Big Architecture Comparison" article. Other non-framed architectures may be covered in future articles.

Note that ideally each of these topics shown in the figure above would deserve at leas whole article itself (and hopefully get it in the future). So, to keep this article at a reason length, many sections are reasonably short. However, I hope this article is still useful as introduction to all the interesting LLM alternatives that emerged in recent years.

PS: The aforementioned PyTorch conference talk will be uploaded to the official PyTorc YouTube channel. In the meantime, if you are curious, you can find a practice recording version below.

0:00

(There is also a YouTube version here.)

1. Transformer-Based LLMs

Transformer-based LLMs based on the classic *Attention Is All You Need* architecture a state-of-the-art across text and code. If we just consider some of the highlights from la 2024 to today, notable models include

- DeepSeek V3/R1
- OLMo 2
- Gemma 3
- Mistral Small 3.1
- Llama 4
- Qwen3
- SmolLM3
- Kimi K2

- gpt-oss
- GLM-4.5
- GLM-4.6
- MiniMax-M2

and many more.

(The list above focuses on the open-weight models; there are proprietary models like C Grok 4, Gemini 2.5, etc. that also fall into this category.)

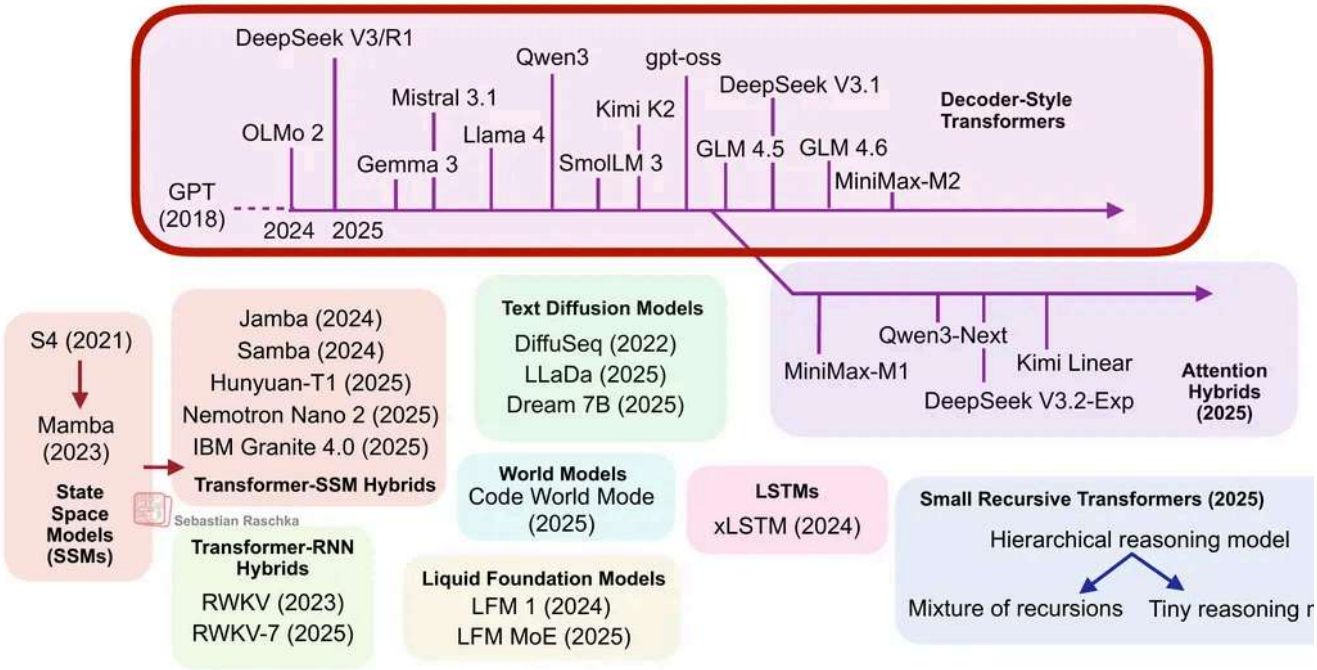


Figure 2: An overview of the most notable decoder-style transformers released in the past year.

Since I talked and wrote about transformer-based LLMs so many times, I assume you a familiar with the broad idea and architecture. If you'd like a deeper coverage, I compare architectures listed above (and shown in the figure below) in my The Big LLM Architec Comparison article.

(Side note: I could have grouped Qwen3-Next and Kimi Linear with the other transform state space model (SSM) hybrids in the overview figure. Personally, I see these other transformer-SSM hybrids as SSMs with transformer components, whereas I see the m

discussed here (Qwen3-Next and Kimi Linear) as transformers with SSM components However, since I have listed IBM Granite 4.0 and NVIDIA Nemotron Nano 2 in the transformer-SSM box, an argument could be made for putting them into a single categ

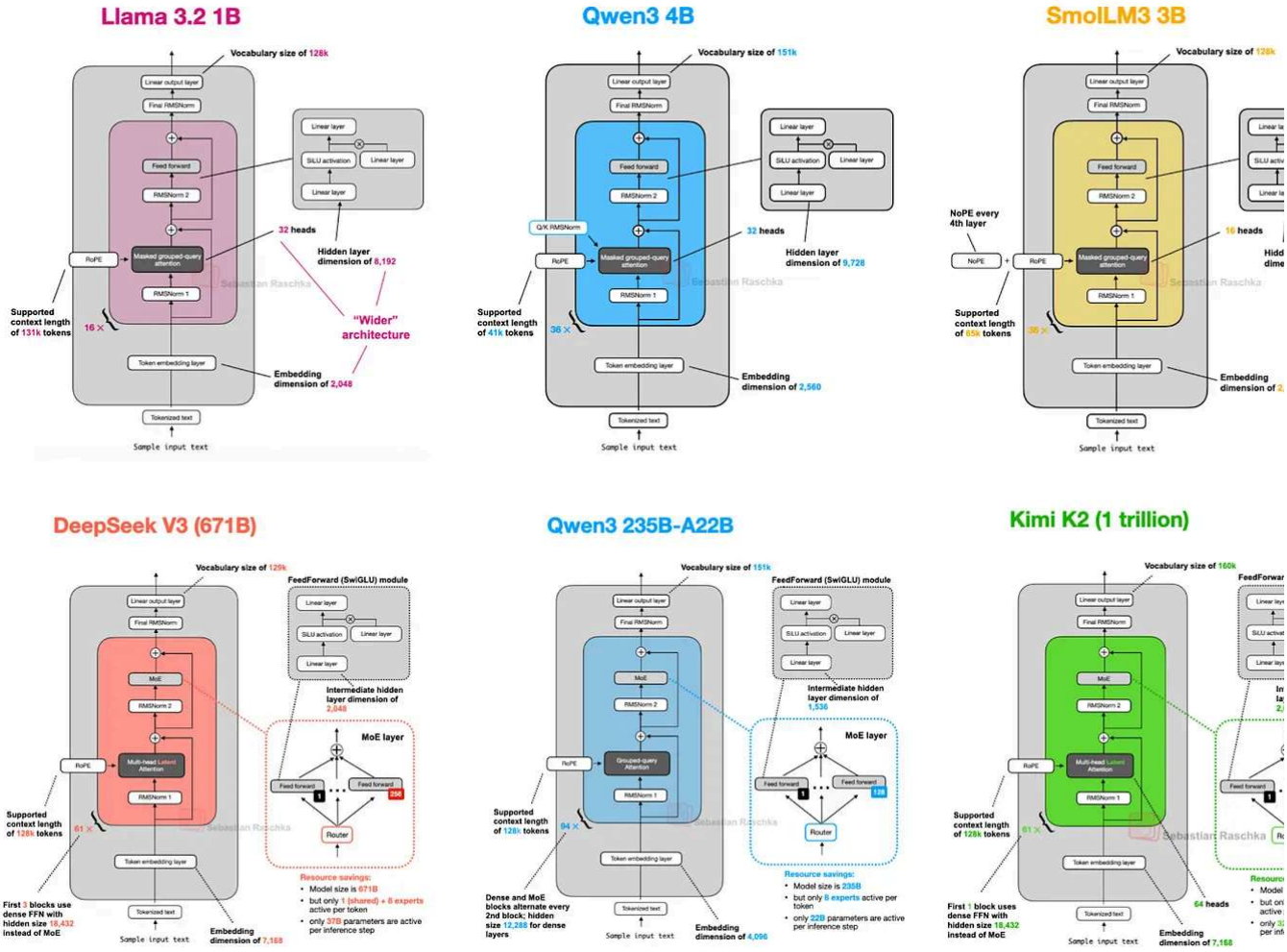


Figure 3. A subset of the architectures discussed in my The Big Architecture Comparison (https://magazine.sebastianraschka.com/p/the-big-llm-architecture-comparison) article.

If you are working with or on LLMs, for example, building applications, fine-tuning mod trying new algorithms, I would make these models my go-to. They are tested, proven, a perform well.

Moreover, as discussed in the *The Big Architecture Comparison* article, there are man efficiency improvements, including grouped-query attention, sliding-window attention head latent attention, and others.

However, it would be boring (and shortsighted) if researchers and engineers didn't wor trying alternatives. So, the remaining sections will cover some of the interesting alterna

that emerged in recent years.

2. (Linear) Attention Hybrids

Before we discuss the "more different" approaches, let's first look at transformer-basec that have adopted more efficient attention mechanisms. In particular, the focus is on th that scale linearly rather than quadratically with the number of input tokens.

There's recently been a revival in linear attention mechanisms to improve the efficiency LLMs.

The attention mechanism introduced in the Attention Is All You Need paper (2017), aka scaled-dot-product attention, remains the most popular attention variant in today's LL Besides traditional multi-head attention, it's also used in the more efficient flavors like grouped-query attention, sliding window attention, and multi-head latent attention as discussed in my talk.

2.1 Traditional Attention and Quadratic Costs

The original attention mechanism scales quadratically with the sequence length:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

This is because the query (Q), key (K), and value (V) are *n*-by-*d* matrices, where *d* is th embedding dimension (a hyperparameter) and *n* is the sequence length (i.e., the numb tokens).

(You can find more details in my Understanding and Coding Self-Attention, Multi-Head Attention, Causal-Attention, and Cross-Attention in LLMs article)

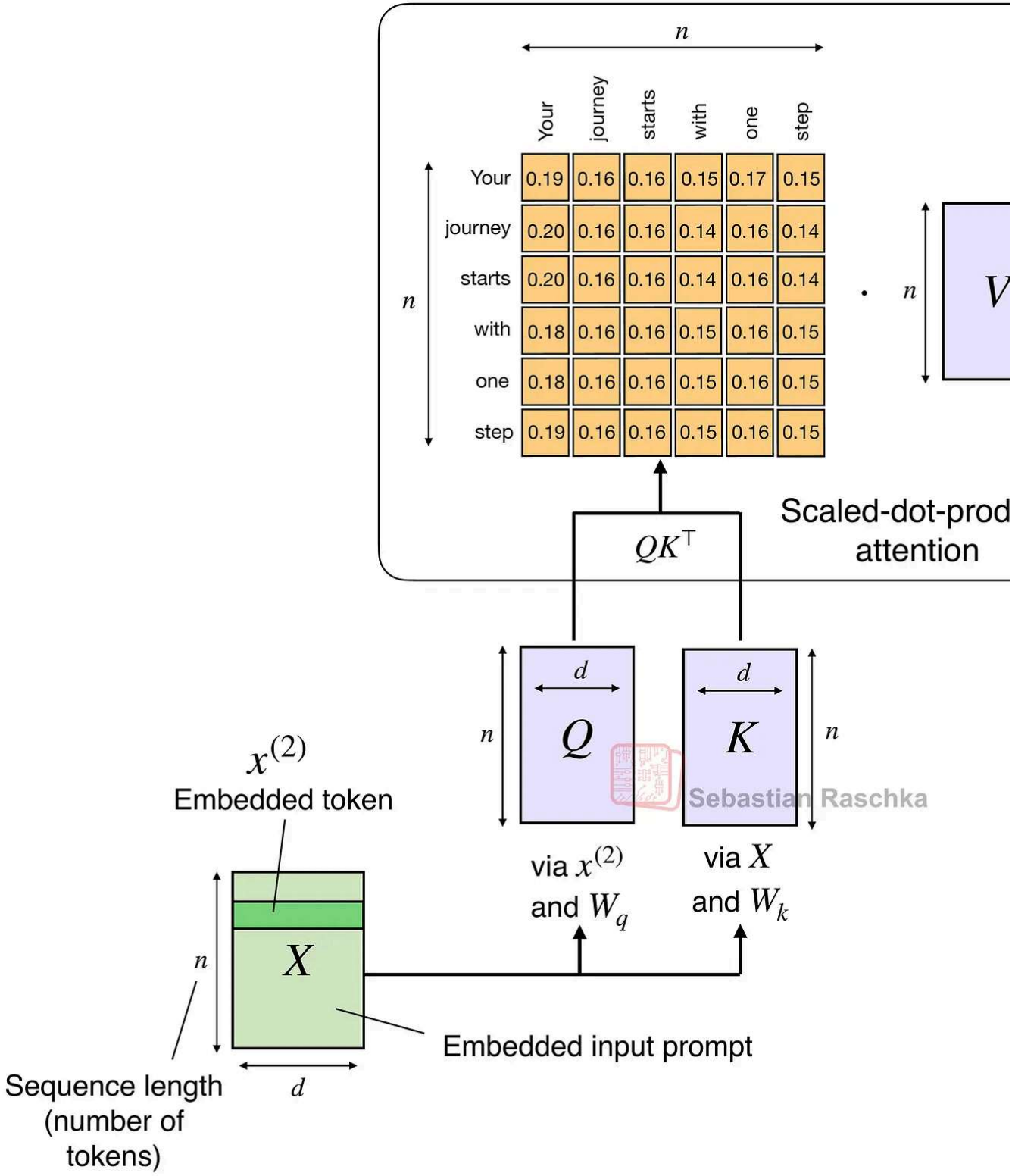


Figure 4: Illustration of the traditional scaled-dot-product attention mechanism in multi-head attention; the quadratic cost in attention due to sequence length n.

2.2 Linear attention

Linear attention variants have been around for a long time, and I remember seeing tons papers in the 2020s. For example, one of the earliest I recall is the 2020 Transformers a

RNNs: Fast Autoregressive Transformers with Linear Attention paper, where the researc approximated the attention mechanism:

$$\text{Attention}(Q,K,V) = \text{softmax}(\frac{QK^\top}{\sqrt{d}})V \approx \phi(Q)(\phi(K)^\top V)$$

Here, $\phi(\cdot)$ is a kernel feature function, set to $\phi(x) = \text{elu}(x)+1$.

This approximation is efficient because it avoids explicitly computing the n×n attentior matrix QK^T.

I don't want to dwell too long on these older attempts. But the bottom line was that the reduced both time and memory complexity from $O(n^2)$ to $O(n)$ to make attention much efficient for long sequences.

However, they never really gained traction as they degraded the model accuracy, and I never really seen one of these variants applied in an open-weight state-of-the-art LLM

2.3 Linear Attention Revival

In the second half of this year, there has been revival of linear attention variants, as well bit of a back-and-forth from some model developers as illustrated in the figure below.

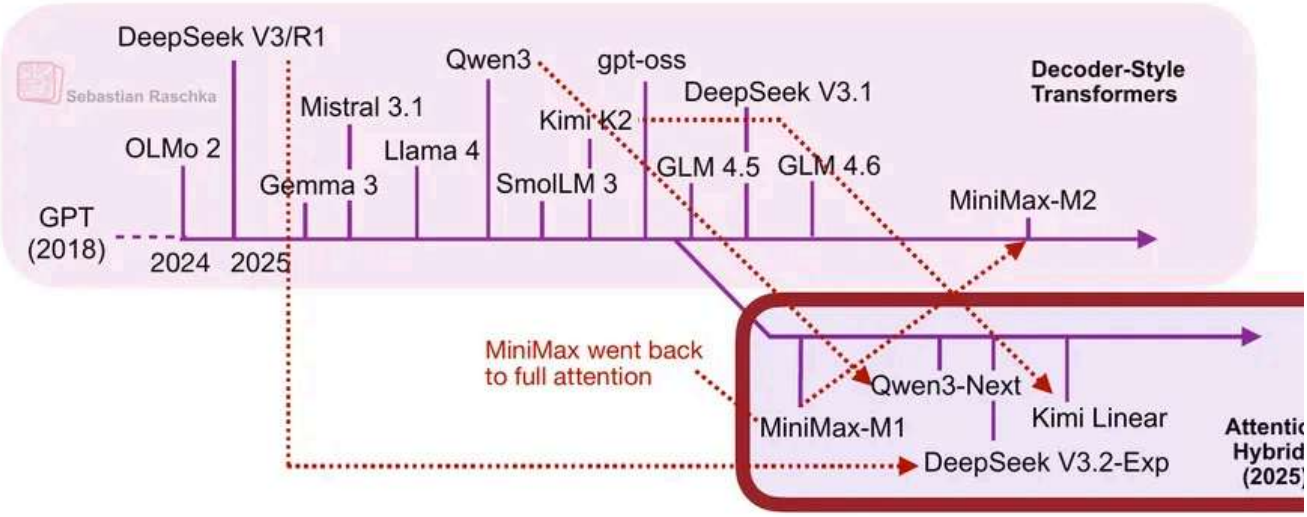


Figure 5: An overview of the linear attention hybrid architectures.

The first notable model was MiniMax-M1 with lightning attention.

MiniMax-M1 is a 456B parameter mixture-of-experts (MoE) model with 46B active parameters, which came out back in June.

Then, in August, the Qwen3 team followed up with Qwen3-Next, which I discussed in detail above. Then, in September, the DeepSeek Team announced DeepSeek V3.2. (DeepSeek V3.2 sparse attention mechanism is not strictly linear but at least subquad terms of computational costs, so I think it's fair to put it into the same category as MiniM M1, Qwen3-Next, and Kimi Linear.)

All three models (MiniMax-M1, Qwen3-Next, DeepSeek V3.2) replace the traditional quadratic attention variants in most or all of their layers with efficient linear variants.

Interestingly, there was a recent plot twist, where the MiniMax team released their new parameter M2 model without linear attention, going back to regular attention. The team stated that linear attention is tricky in production LLMs. It seemed to work fine with reg prompts, but it had poor accuracy in reasoning and multi-turn tasks, which are not only important for regular chat sessions but also agentic applications.

This could have been a turning point where linear attention may not be worth pursuing all. However, it gets more interesting. In October, the Kimi team released their new Kimi model with linear attention.

For this linear attention aspect, both Qwen3-Next and Kimi Linear adopt a Gated Delta which I wanted to discuss in the next few sections as one example of a hybrid attentior architecture.

2.4 Qwen3-Next

Let's start with Qwen3-Next, which replaced the regular attention mechanism by a Gat DeltaNet + Gated Attention hybrid, which helps enable the native 262k token context l in terms of memory usage (the previous 235B-A22B model model supported 32k nati and 131k with YaRN scaling.)

Their hybrid mechanism mixes Gated DeltaNet blocks with Gated Attention blocks wit 3:1 ratio as shown in the figure below.

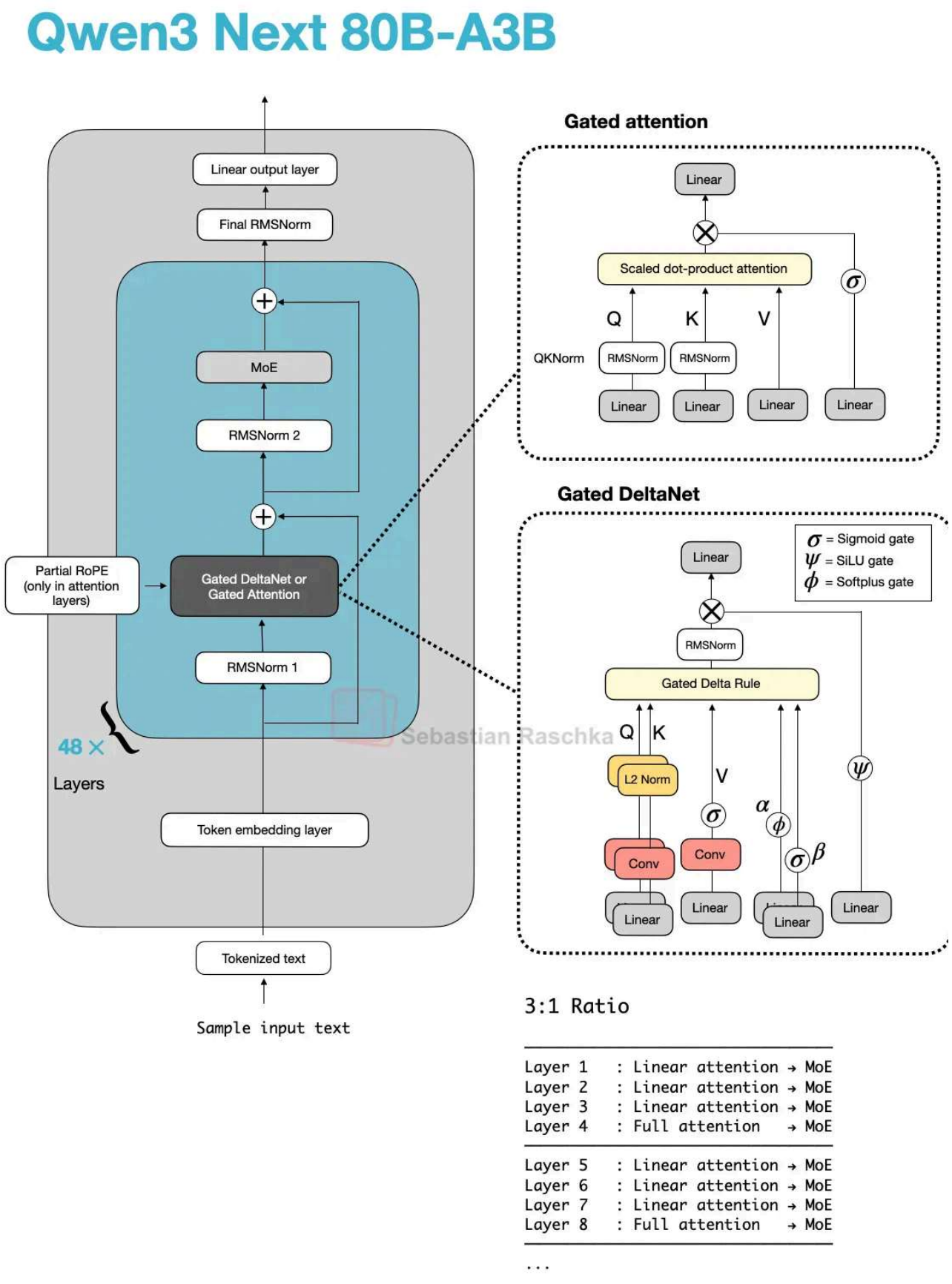


Figure 6: Qwen3-Next with gated attention and Gated DeltaNet.

As depicted in the figure above, the attention mechanism is either implemented as gat attention or Gated DeltaNet. This simply means the 48 transformer blocks (layers) in th architecture alternate between this. Specifically, as mentioned earlier, they alternate in a ratio. For instance, the transformer blocks are as follows:

```
————————————————————————————————————————————————————
Layer 1 : Linear attention → MoE
Layer 2 : Linear attention → MoE
Layer 3 : Linear attention → MoE
Layer 4 : Full attention → MoE
————————————————————————————————————————————————————
Layer 5 : Linear attention → MoE
Layer 6 : Linear attention → MoE
Layer 7 : Linear attention → MoE
Layer 8 : Full attention → MoE
————————————————————————————————————————————————————
...
```

Otherwise, the architecture is pretty standard and similar to Qwen3:

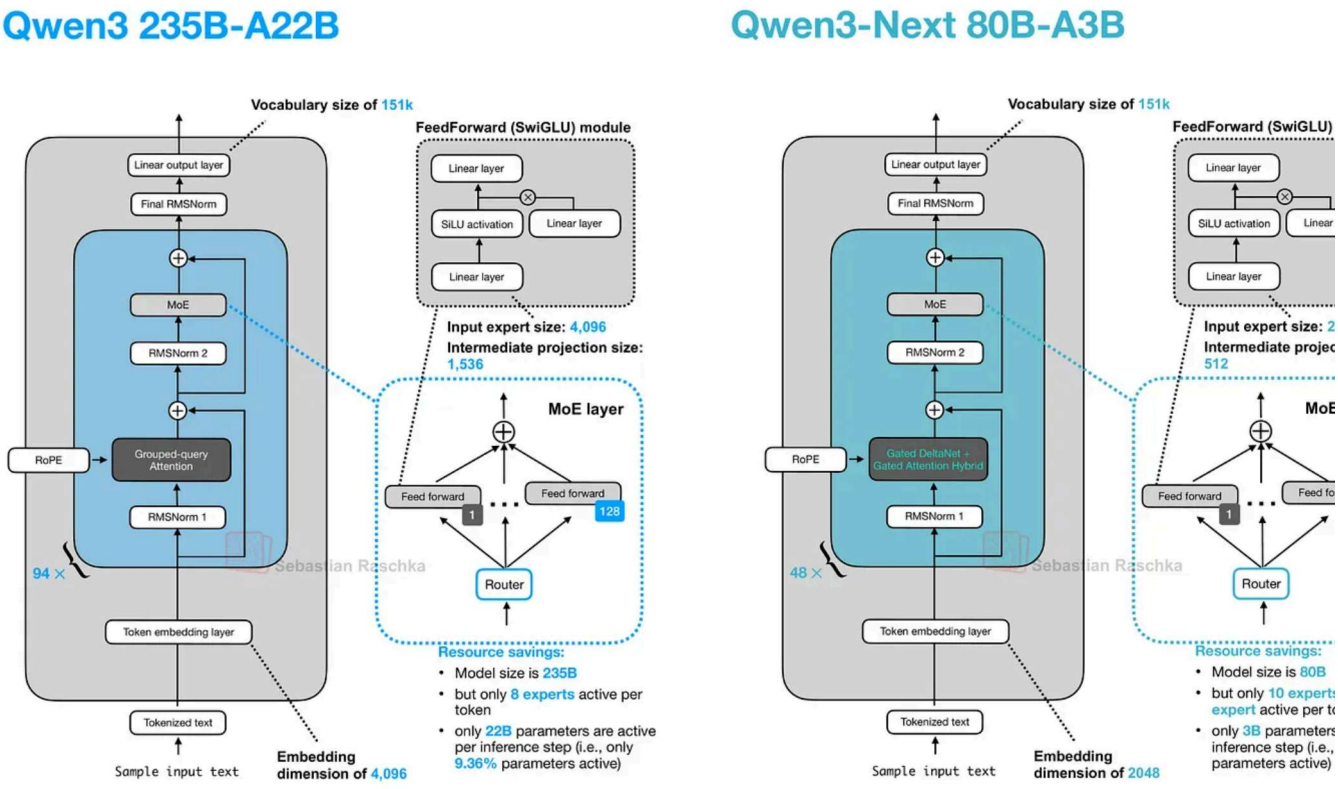


Figure 7: A previous "regular" Qwen3 model (left) next to Qwen3-Next (right).

So, what are gated attention and Gated DeltaNet?

2.5 Gated Attention

Before we get to the Gated DeltaNet itself, let's briefly talk about the gate. As you can s the upper part of the Qwen3-Next architecture in the previous figure, Qwen3-Next use "gated attention". This is essentially regular full attention with an additional sigmoid gat

This gating is a simple modification that I added to an `MultiHeadAttention` implemen (based on code from chapter 3 of my LLMs from Scratch book) below for illustration purposes:

```python
import torch
from torch import nn

class GatedMultiHeadAttention(nn.Module):
    def __init__(
        self, d_in, d_out, context_length, dropout, num_heads, qkv_bias=False
    ):
        super().__init__()
        assert d_out % num_heads == 0

        self.d_out = d_out
        self.num_heads = num_heads
        self.head_dim = d_out // num_heads

        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        ####################################################
        ### NEW: Add gate
        self.W_gate = nn.Linear(d_in, d_out, bias=qkv_bias)
        ####################################################
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)

        self.out_proj = nn.Linear(d_out, d_out)
        self.dropout = nn.Dropout(dropout)

        self.register_buffer(
            "mask",
            torch.triu(torch.ones(context_length, context_length), diagonal=1
            persistent=False,
        )

    def forward(self, x):
        b, num_tokens, _ = x.shape
        queries = self.W_query(x)
        ####################################################
        ### NEW: Add gate
        gate = self.W_gate(x)
        ####################################################
        keys = self.W_key(x)
        values = self.W_value(x)

        keys = keys.view(b, num_tokens, self.num_heads, self.head_dim)
        values = values.view(b, num_tokens, self.num_heads, self.head_dim)
        queries = queries.view(b, num_tokens, self.num_heads, self.head_dim)

        keys = keys.transpose(1, 2)
        queries = queries.transpose(1, 2)
        values = values.transpose(1, 2)
```

```
49
50          attn_scores = queries @ keys.transpose(2, 3)
51
52          mask_bool = self.mask.bool()[:num_tokens, :num_tokens]
53          attn_scores.masked_fill_(
54              mask_bool, torch.finfo(attn_scores.dtype).min
55          )
56
57          attn_weights = torch.softmax(
58              attn_scores / (self.head_dim ** 0.5), dim=-1
59          )
60          attn_weights = self.dropout(attn_weights)
61
62          context = (attn_weights @ values).transpose(1, 2)
63          context = context.reshape(b, num_tokens, self.d_out)
64
65          ############################################################
66          ### NEW: Add gate
67          context = context * torch.sigmoid(gate)
68          ############################################################
69          out = self.out_proj(context)
70          return out
```

As we can see, after computing attention as usual, the model uses a separate gating si from the same input, applies a sigmoid to keep it between 0 and 1, and multiplies it with attention output. This allows the model to scale up or down certain features dynamical Qwen3-Next developers state that this helps with training stability:

> [...] the attention output gating mechanism helps eliminate issues like Attention Sinl Massive Activation, ensuring numerical stability across the model.

In short, gated attention modulates the *output* of standard attention. In the next sectior discuss Gated DeltaNet, which replaces the attention mechanism itself with a recurren delta-rule memory update.

2.6 Gated DeltaNet

Now, what is Gated DeltaNet? Gated DeltaNet (short for *Gated Delta Network*) is Qwe Next's linear-attention layer, which is intended as an alternative to standard softmax attention. It was adopted from the Gated Delta Networks: Improving Mamba2 with Delt paper as mentioned earlier.

Gated DeltaNet was originally proposed as an improved version of Mamba2, where it combines the gated decay mechanism of Mamba2 with a delta rule.

Mamba is a state-space model (an alternative to transformers), a big topic that deserve separate coverage in the future.

The delta rule part refers to computing the difference (delta, Δ) between new and predi values to update a hidden state that is used as a memory state (more on that later).

(Side note: Readers with classic machine learning literature can think of this as similar t Hebbian learning inspired by biology: "Cells that fire together wire together." It's basica precursor of the perceptron update rule and gradient descent-based learning, but with supervision.)

Gated DeltaNet has a gate similar to the gate in gated attention discussed earlier, exce it uses a SiLU instead of logistic sigmoid activation, as illustrated below. (The SiLU cho likely to improve gradient flow and stability over the standard sigmoid.)

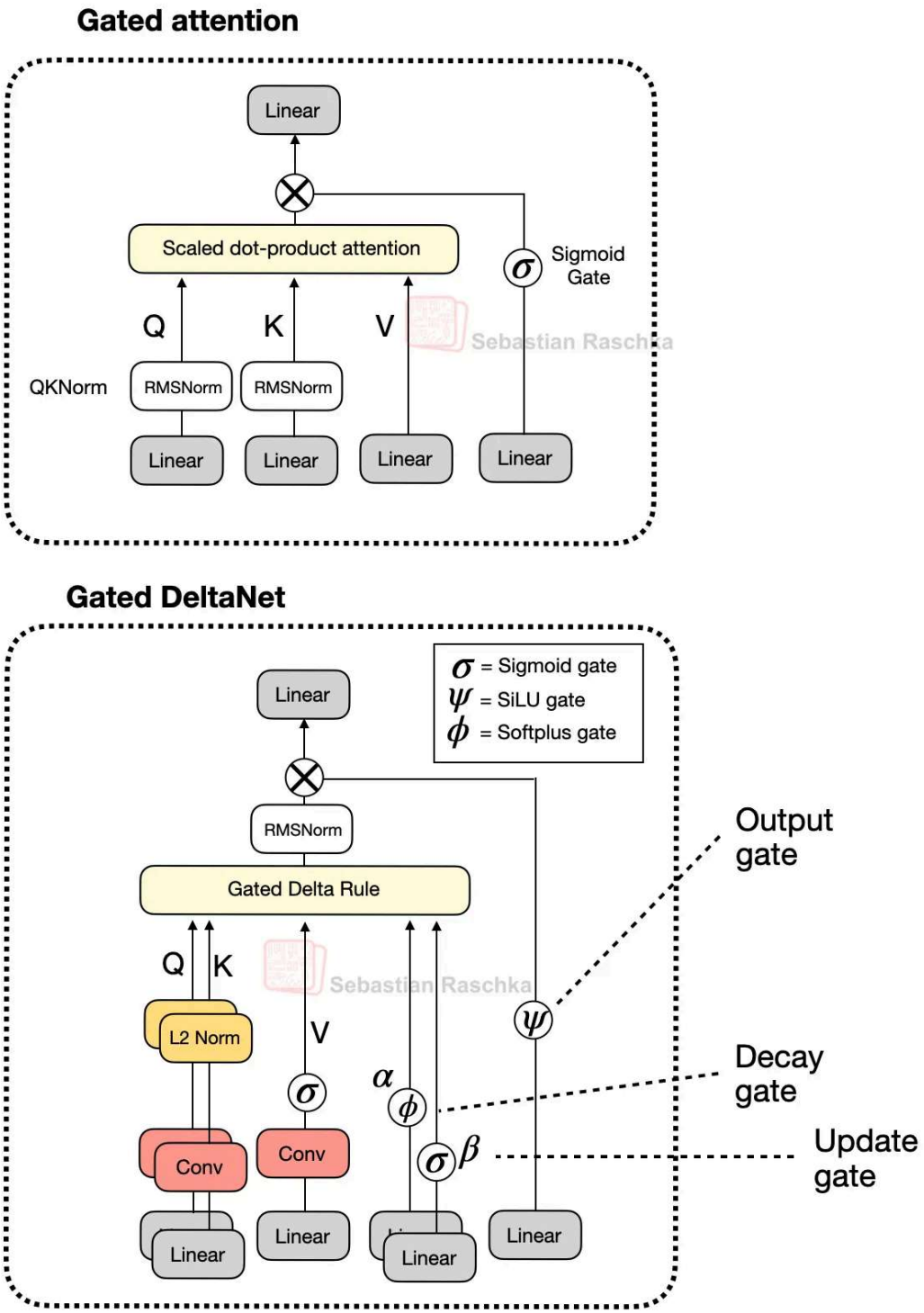


Figure 8: Gated attention compared to Gated DeltaNet.

However, as shown in the figure above, next to the output gate, the "gated" in the Gated DeltaNet also refers to several additional gates:

- α (decay gate) controls how fast the memory decays or resets over time,
- β (update gate) controls how strongly new inputs modify the state.

In code, a simplified version of the Gated DeltaNet depicted above (without the convol mixing) can be implemented as follows (the code is inspired by the official implementa the Qwen3 team):

```python
import torch
from torch import nn
import torch.nn.functional as F

def l2norm(x, dim=-1, eps=1e-6):
    return x * torch.rsqrt((x * x).sum(dim=dim, keepdim=True) + eps)

class GatedDeltaNet(nn.Module):
    def __init__(
        self, d_in, d_out, dropout, num_heads, qkv_bias=False
    ):
        super().__init__()
        assert d_out % num_heads == 0

        self.d_out = d_out
        self.num_heads = num_heads
        self.head_dim = d_out // num_heads

        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        ##################################################
        ### NEW: Gates for delta rule and output gating
        self.W_gate = nn.Linear(d_in, d_out, bias=False)
        self.W_beta = nn.Linear(d_in, d_out, bias=False)

        # Note: The decay gate alpha corresponds to
        # A_log + W_alpha(x) + dt_bias
        self.W_alpha = nn.Linear(d_in, num_heads, bias=False)
        self.dt_bias = nn.Parameter(torch.ones(num_heads))
        self.A_log = nn.Parameter(torch.zeros(num_heads))
        # We could implement this as
        # W_alpha = nn.Linear(d_in, num_heads, bias=True)
        # but the bias is separate for interpretability and
        # to mimic the official implementation

        self.norm = nn.RMSNorm(self.head_dim, eps=1e-6)
        ##################################################

        self.out_proj = nn.Linear(d_out, d_out)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        b, num_tokens, _ = x.shape
        queries = self.W_query(x)
        keys = self.W_key(x)
```

```
        values = self.W_value(x)
        ##################################################
        ### NEW: Compute delta rule gates
        beta = torch.sigmoid(self.W_beta(x))
        alpha = -self.A_log.exp().view(1, 1, -1) * F.softplus(
            self.W_alpha(x) + self.dt_bias
        )
        gate = self.W_gate(x)
        ##################################################

        keys = keys.view(b, num_tokens, self.num_heads, self.head_dim)
        values = values.view(b, num_tokens, self.num_heads, self.head_dim)
        queries = queries.view(b, num_tokens, self.num_heads, self.head_di
        beta = beta.view(b, num_tokens, self.num_heads, self.head_dim)
        gate = gate.view(b, num_tokens, self.num_heads, self.head_dim)

        keys = keys.transpose(1, 2)
        queries = queries.transpose(1, 2)
        values = values.transpose(1, 2)
        beta = beta.transpose(1, 2)
        gate = gate.transpose(1, 2)

        ##################################################
        ### NEW: QKNorm-like normalization for delta rule
        queries = l2norm(queries, dim=-1) / (self.head_dim ** 0.5)
        keys = l2norm(keys, dim=-1)
        ##################################################

        S = x.new_zeros(b, self.num_heads, self.head_dim, self.head_dim)

        outs = []
        ##################################################
        ### NEW: Gated delta rule update
        for t in range(num_tokens):
            k_t = keys[:, :, t]
            q_t = queries[:, :, t]
            v_t = values[:, :, t]
            b_t = beta[:, :, t]
            a_t = alpha[:, t].unsqueeze(-1).unsqueeze(-1)

            S = S * a_t.exp()
            kv_mem = (S * k_t.unsqueeze(-1)).sum(dim=-2)
            delta = (v_t - kv_mem) * b_t
            S = S + k_t.unsqueeze(-1) * delta.unsqueeze(-2)
            y_t = (S * q_t.unsqueeze(-1)).sum(dim=-2)
            ##################################################
            outs.append(y_t)

        context = torch.stack(outs, dim=2).transpose(1, 2).contiguous()
        context = context.view(b, num_tokens, self.num_heads, self.head_d
```

```
97
98          ##################################################
99          ### NEW: Apply RMSNorm and SiLU gate
100         context = self.norm(context)
101         context = context * F.silu(gate)
102         ##################################################
103
104         context = context.view(b, num_tokens, self.d_out)
105         context = self.dropout(context)
106         out = self.out_proj(context)
107         return out
108
109
```

(Note that for simplicity, I omitted the convolutional mixing that Qwen3-Next and Kimi use to keep the code more readable and focus on the recurrent aspects.)

So, as we can see above, there are lots of differences to standard (or gated) attention.

In gated attention, the model computes normal attention between all tokens (every tok attends or looks at every other token). Then, after getting the attention output, a gate (a sigmoid) decides how much of that output to keep. The takeaway is that it's still the reg scaled-dot product attention that scales quadratically with the context length.

As a refresher, scaled-dot product attention is computed as softmax(QK^T)V, where Q a are *n*-by-*d* matrices, where *n* is the number of input tokens, and *d* is the embedding dimension. So QK^T results in an attention *n*-by-*n* matrix, that is multiplied by an *n*-by-*d* dimensional value matrix *V*.

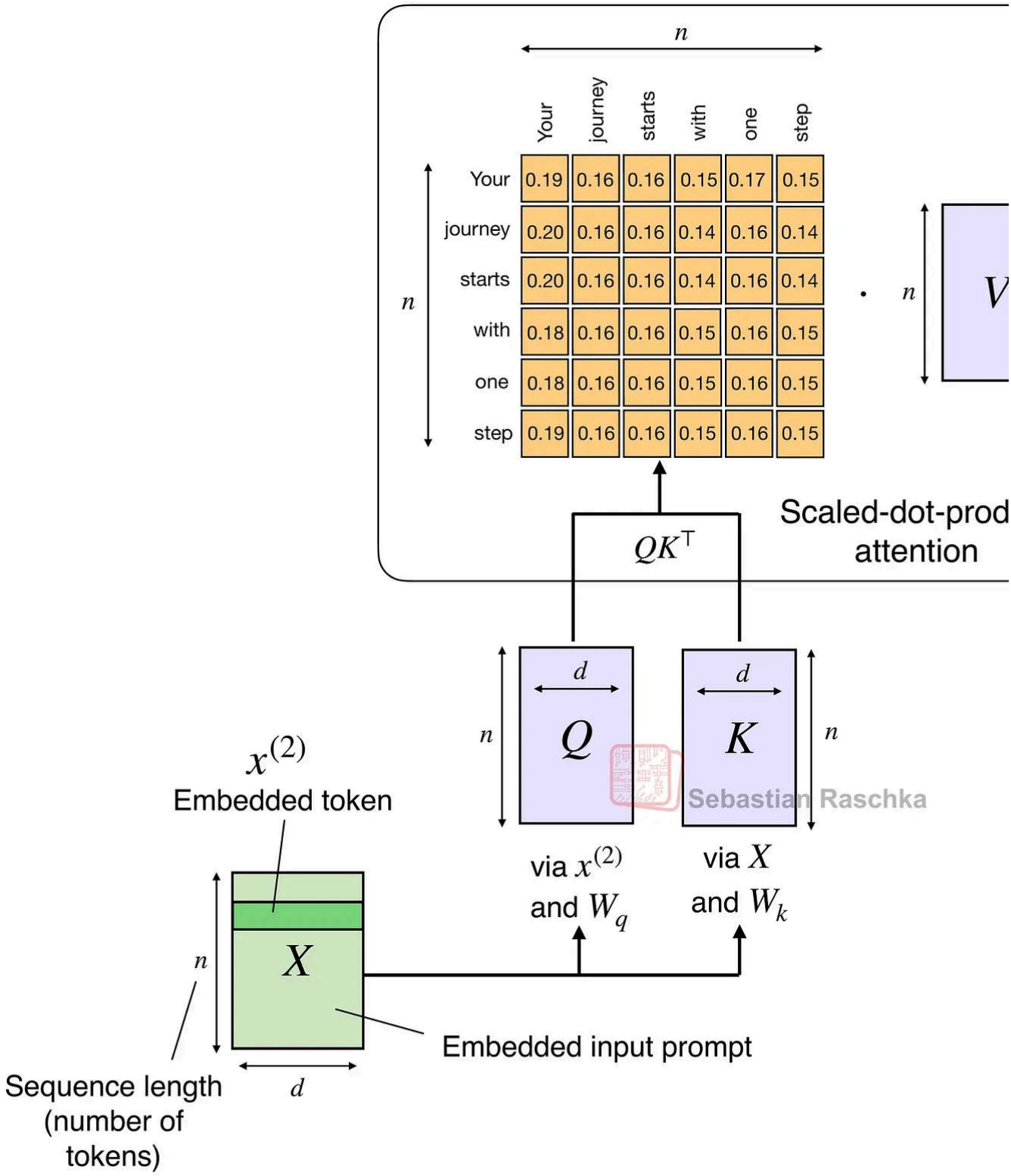


Figure 9: The traditional attention mechanism (again), which scales with the number of tokens *n*.

In Gated DeltaNet, there's no *n*-by-*n* attention matrix. Instead, the model processes tok one by one. It keeps a running memory (a state) that gets updated as each new token c in. This is what's implemented as, where S is the state that gets updated recurrently for time step *t*.

```
S = x.new_zeros(b, self.num_heads, self.head_dim, self.head_dim)
outs = []

for t in range(num_tokens):
    k_t = keys[:, :, t]
    q_t = queries[:, :, t]
    v_t = values[:, :, t]
    b_t = beta[:, :, t]
    a_t = alpha[:, t].unsqueeze(-1).unsqueeze(-1)

    S = S * a_t.exp()
    kv_mem = (S * k_t.unsqueeze(-1)).sum(dim=-2)
    delta = (v_t - kv_mem) * b_t
    S = S + k_t.unsqueeze(-1) * delta.unsqueeze(-2)
    y_t = (S * q_t.unsqueeze(-1)).sum(dim=-2)
```

And the gates control how that memory changes:

- α (alpha) regulates how much of the old memory to forget (decay).
- β (beta) regulates how much the current token at time step *t* updates the memory.

(And the final output gate, not shown in the snippet above, is similar to gated attention; controls how much of the output is kept.)

So, in a sense, this state update in Gated DeltaNet is similar to how recurrent neural net (RNNs) work. The advantage is that it scales linearly (via the for-loop) instead of quadratically with context length.

The downside of this recurrent state update is that, compared to regular (or gated) atte it sacrifices the global context modeling ability that comes from full pairwise attention.

Gated DeltaNet, can, to some extend, still capture context, but it has to go through the memory (*S*) bottleneck. That memory is a fixed size and thus more efficient, but it compresses past context into a single hidden state similar to RNNs.

That's why the Qwen3-Next and Kimi Linear architectures don't replace all attention la with DeltaNet layers but use the 3:1 ratio mentioned earlier.

2.7 DeltaNet Memory Savings

In the previous section, we discussed the advantage of the DeltaNet over full attention terms of linear instead of quadratic compute complexity with respect to the context ler

Next to the linear compute complexity, another big advantage of DeltaNet is the memc savings, as DeltaNet modules don't grow the KV cache. (For more information about K caching, see my Understanding and Coding the KV Cache in LLMs from Scratch articl Instead, as mentioned earlier, they keep a fixed-size recurrent state, so memory stays constant with context length.

For a regular multi-head attention (MHA) layer, we can compute the KV cache size as follows:

```
KV_cache_MHA ≈ batch_size × n_tokens × n_heads × d_head × 2 × bytes
```

(The 2 multiplier is there because we have both keys and values that we store in the ca

For the simplified DeltaNet version implemented above, we have:

```
KV_cache_DeltaNet = batch_size × n_heads × d_head × d_head × bytes
```

Note that the `KV_cache_DeltaNet` memory size doesn't have a context length (`n_toke` dependency. Also, we have only the memory state S that we store instead of separate l and values, hence `2 × bytes` becomes just bytes. However, note that we now have a quadratic `d_head × d_head` in here. This comes from the state:

```
S = x.new_zeros(b, self.num_heads, self.head_dim, self.head_dim)
```

But that's usually nothing to worry about, as the head dimension is usually relatively sm instance, it's 128 in Qwen3-Next.

The full version with the convolutional mixing is a bit more complex, including the kerne and so on, but the formulas above should illustrate the main trend and motivation behir Gated DeltaNet.

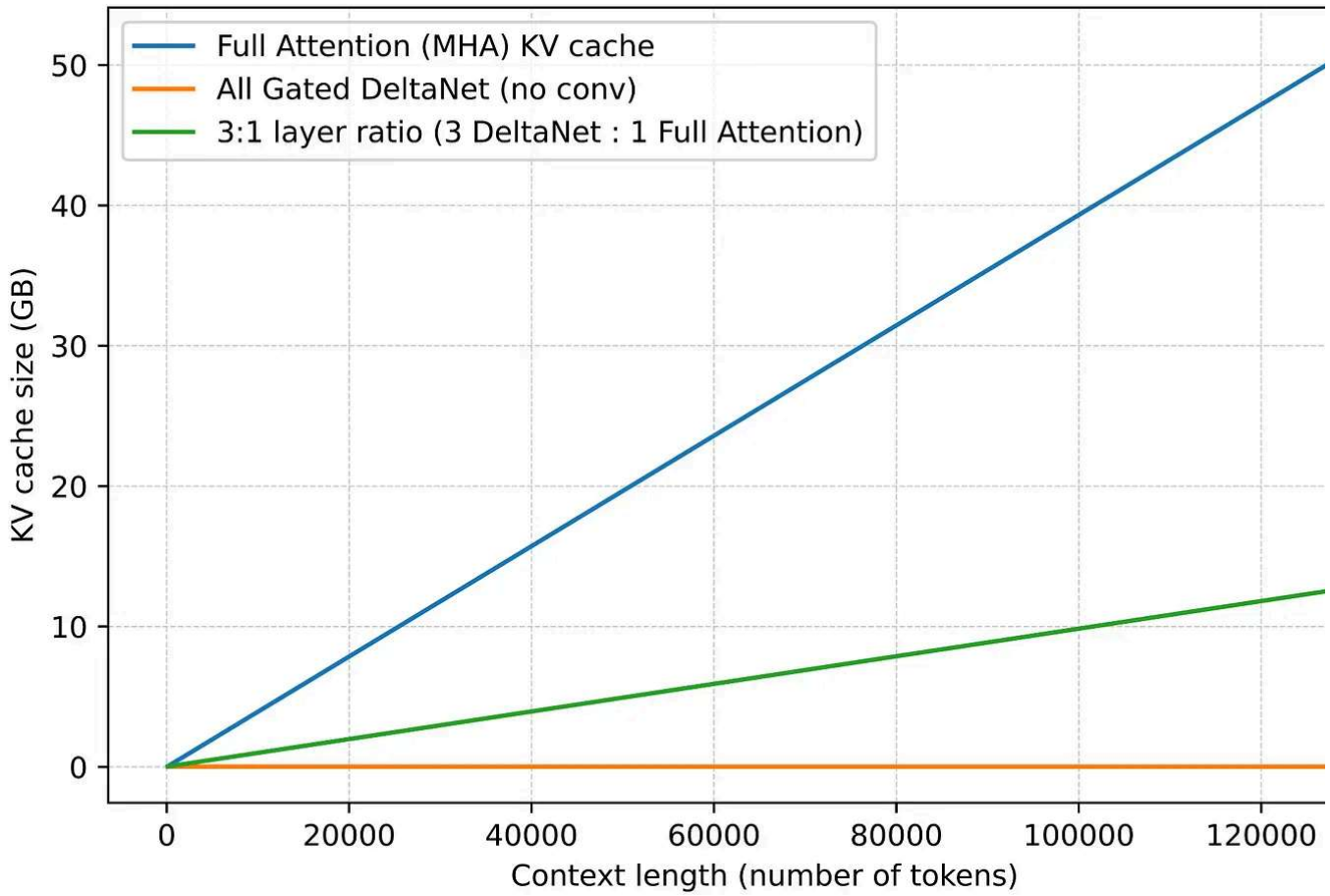


Figure 10: A comparison of the growing KV cache size. The 3:1 ratio refers to the ratio of Gated DeltaNet to full attention layers. The calculation assumes emb_dim=2048, n_heads=16, n_layers=48, bf16. You can find the code to reproduce this here: https://github.com/rasbt/LLMs-from-scratch/tree/main/ch04/08_deltanet.

2.8 Kimi Linear vs. Qwen3-Next

Kimi Linear shares several structural similarities with Qwen3-Next. Both models rely or hybrid attention strategy. Concretely, they combine lightweight linear attention with he full attention layers. Specifically, both use a 3:1 ratio, meaning for every three transform blocks employing the linear Gated DeltaNet variant, there's one block that uses full atte as shown in the figure below.

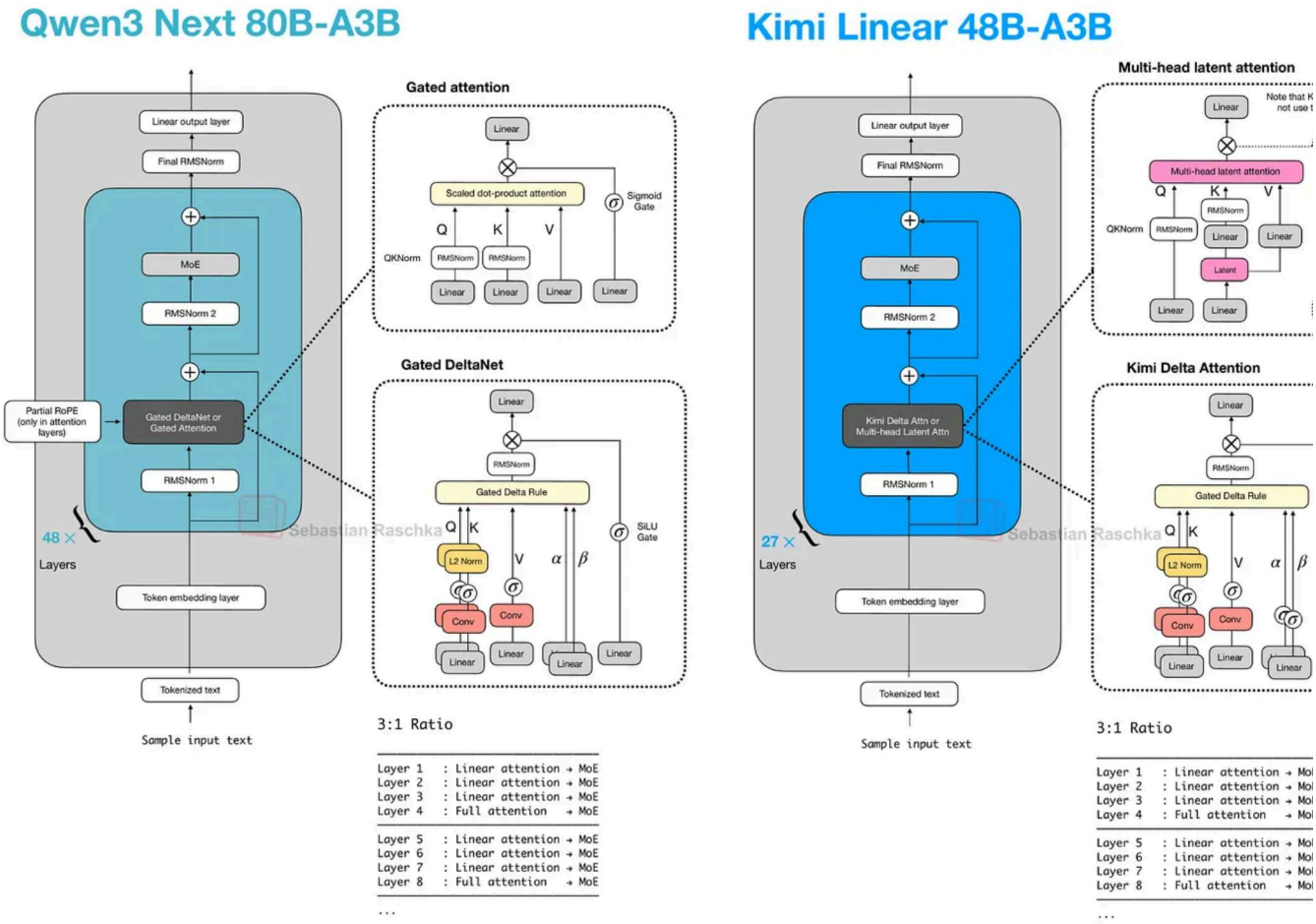


Figure 11: Qwen3-Next and Kimi Linear side by side.

Gated DeltaNet is a linear attention variant with inspiration from recurrent neural netwc including a gating mechanism from the Gated Delta Networks: Improving Mamba2 wit Delta Rule paper. In a sense, Gated DeltaNet is a DeltaNet with Mamba-style gating, ar DeltaNet is a linear attention mechanism (more on that in the next section)

The MLA in Kimi Linear, depicted in the upper right box in the Figure 11 above, does no the sigmoid gate.This omission was intentional so that the authors could compare the architecture more directly to standard MLA, however, they stated that they plan to add the future.

Also note that the omission of the RoPE box in the Kimi Linear part of the figure above intentional as well. Kimi applies NoPE (No Positional Embedding) in multi-head latent attention MLA) layers (global attention). As the authors state, this lets MLA run as pure query attention at inference and avoids RoPE retuning for long-context scaling (the po bias is supposedly handled by the Kimi Delta Attention blocks). For more information o

and multi-query attention, which is a special case of grouped-query attention, please s
The Big LLM Architecture Comparison article.

2.9 Kimi Delta Attention

Kimi Linear modifies the linear attention mechanism of Qwen3-Next by the Kimi Delta Attention (KDA) mechanism, which is essentially a refinement of Gated DeltaNet. Whereas Qwen3-Next applies a scalar gate (one value per attention head) to control tl memory decay rate, Kimi Linear replaces it with a channel-wise gating for each feature dimension. According to the authors, this gives more control over the memory, and this turn, improves long-context reasoning.

In addition, for the full attention layers, Kimi Linear replaces Qwen3-Next's gated attent layers (which are essentially standard multi-head attention layers with output gating) v multi-head latent attention (MLA). This is the same MLA mechanism used by DeepSee V3/R1 (as discussed in my The Big LLM Architecture Comparison article) but with an additional gate. (To recap, MLA compresses the key/value space to reduce the KV cach size.)

There's no direct comparison to Qwen3-Next, but compared to the Gated DeltaNet-H model from the Gated DeltaNet paper (which is essentially Gated DeltaNet with sliding window attention), Kimi Linear achieves higher modeling accuracy while maintaining th same token-generation speed.

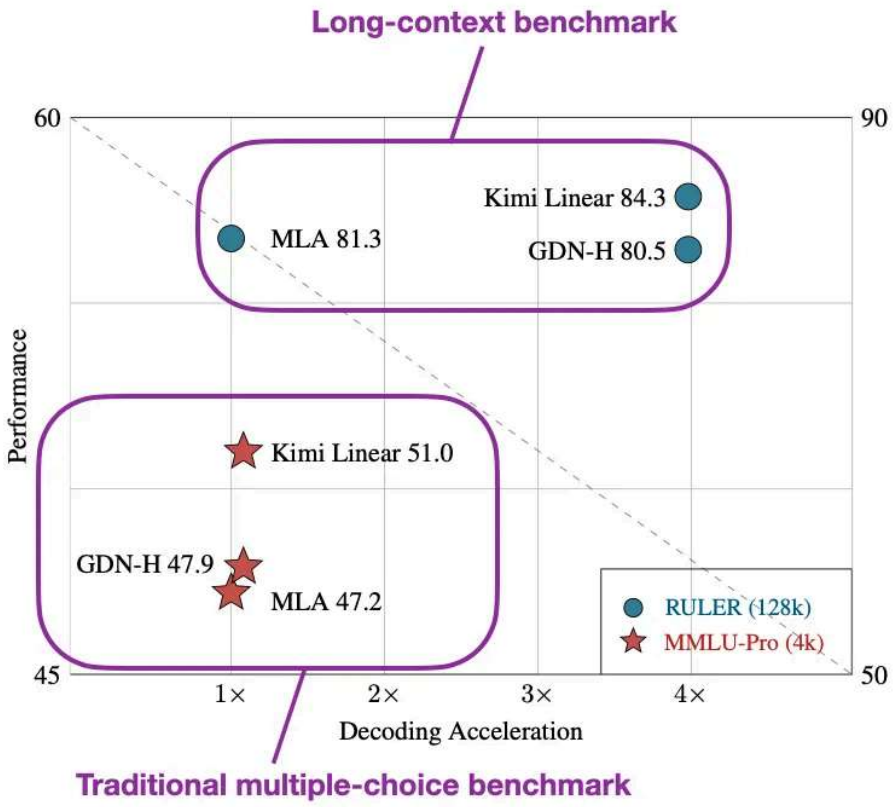


Figure 12: Annotated figure from the Kimi Linear paper (https://arxiv.org/abs/2510.26692) showing that Kimi Linear is as fast as GatedDeltaNet, and much faster than an architecture with multi-head latent attention (like DeepSeek V3/R1), while having a higher benchmark performance.

Furthermore, according to the ablation studies in the DeepSeek-V2 paper, MLA is on p regular full attention when the hyperparameters are carefully chosen.

And the fact that Kimi Linear compares favorably to MLA on long-context and reasonir benchmarks makes linear attention variant once again promising for larger state-of-the models. That being said, Kimi Linear is 48B-parameter large, but it's 20x smaller than K K2. It will be interesting to see if the Kimi team adopts this approach for their upcoming model.

2.10 The Future of Attention Hybrids

Linear attention is not a new concept, but the recent revival of hybrid approaches show researchers are again seriously looking for practical ways to make transformers more efficient. For example Kimi Linear, compared to regular full attention, has a 75% KV cac reduction and up to 6x decoding throughput.

What makes this new generation of linear attention variants different from earlier attem that they are now used together with standard attention rather than replacing it comple

Looking ahead, I expect that the next wave of attention hybrids will focus on further improving long-context stability and reasoning accuracy so that they get closer to the attention state-of-the-art.

3. Text Diffusion Models

A more radical departure from the standard autoregressive LLM architecture is the fam text diffusion models.

You are probably familiar with diffusion models, which are based on the Denoising Diffu Probabilistic Models paper from 2020 for generating images (as a successor to genera adversarial networks) that was later implemented, scaled, and popularized by Stable Diffusion and others.

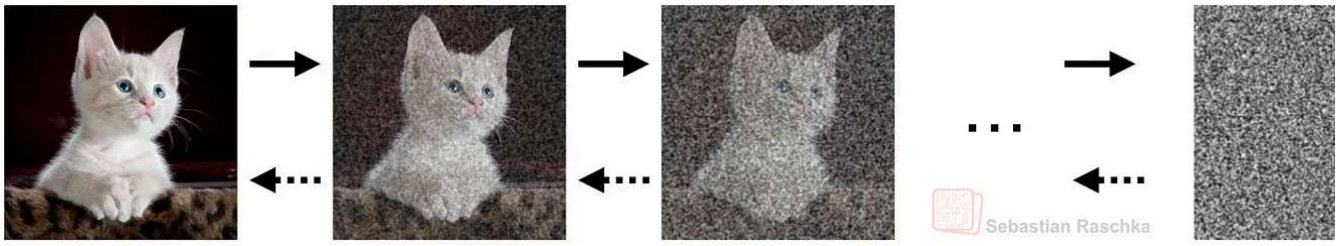


Figure 13: Illustration of an image diffusion process from my very first Substack article in 2022. Here, Gaussian noise is added from left to right, and the model's task is to learn how to remove the noise (from right to left).

3.1 Why Work on Text Diffusion?

With the Diffusion-LM Improves Controllable Text Generation paper in 2022, we also s to see the beginning of a trend where researchers started to adopt diffusion models fo generating text. And I've seen a whole bunch of text diffusion papers in 2025. When I ju checked my paper bookmark list, there are 39 text diffusion models on there! Given th popularity of these models, I thought it was finally time to talk about them.

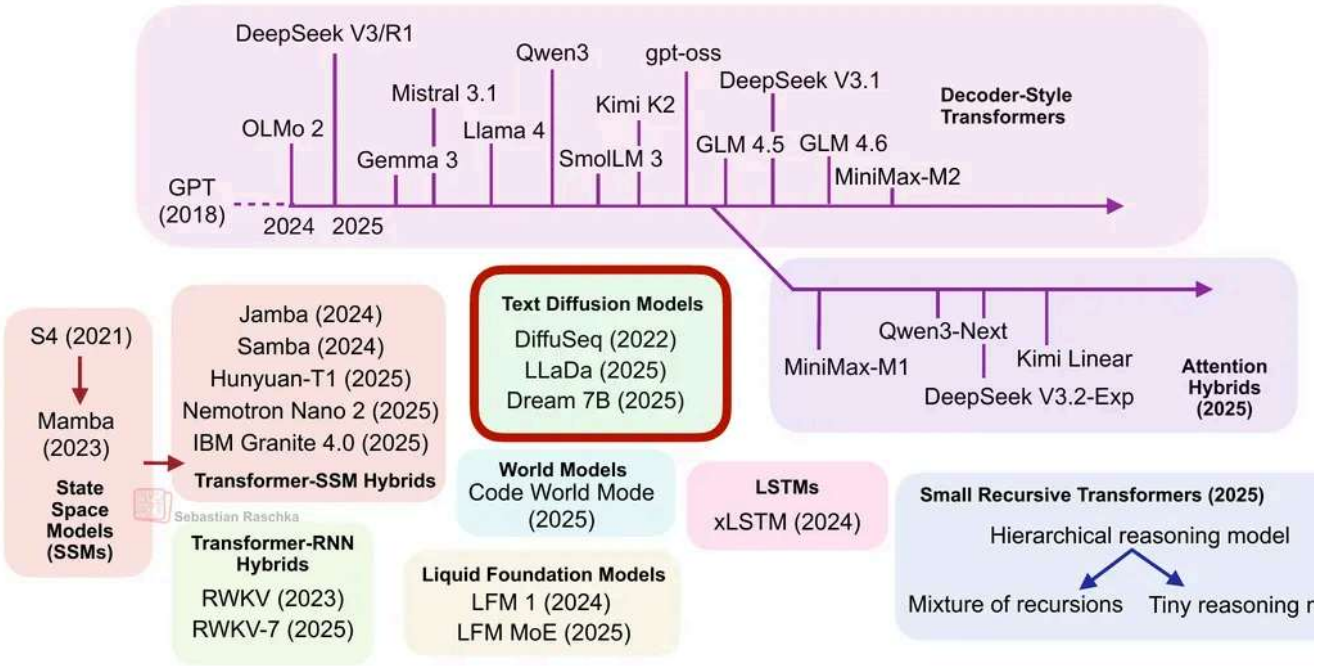


Figure 14: This section covers text diffusion models.

So, what's the advantage of diffusion models, and why are researchers looking into this alternative to traditional, autoregressive LLMs?

Traditional transformer-based (autoregressive) LLMs generate one token at a time. For brevity, let's refer to them simply as *autoregressive LLMs*. Now, the main selling point o *diffusion-based LLMs* (let's call them "diffusion LLMs") is that they can generate multi tokens in parallel rather than sequentially.

Note that diffusion LLMs still require multiple denoising steps. However, even if a diffus model needs, say, 64 denoising steps to produce all tokens in parallel at each step, this computationally more efficient than performing 2,000 sequential generation steps to produce a 2,000-token response.

3.2 The Denoising Process

The denoising process in a diffusion LLM, analogous to the denoising process in regul image diffusion models, is shown in the GIF below. (The key difference is that, instead adding Gaussian noise to pixels, text diffusion corrupts sequences by masking tokens probabilistically.)

For this experiment, I ran the 8B instruct model from the Large Language Diffusion Mo (LLaDA) paper that came out earlier this year.

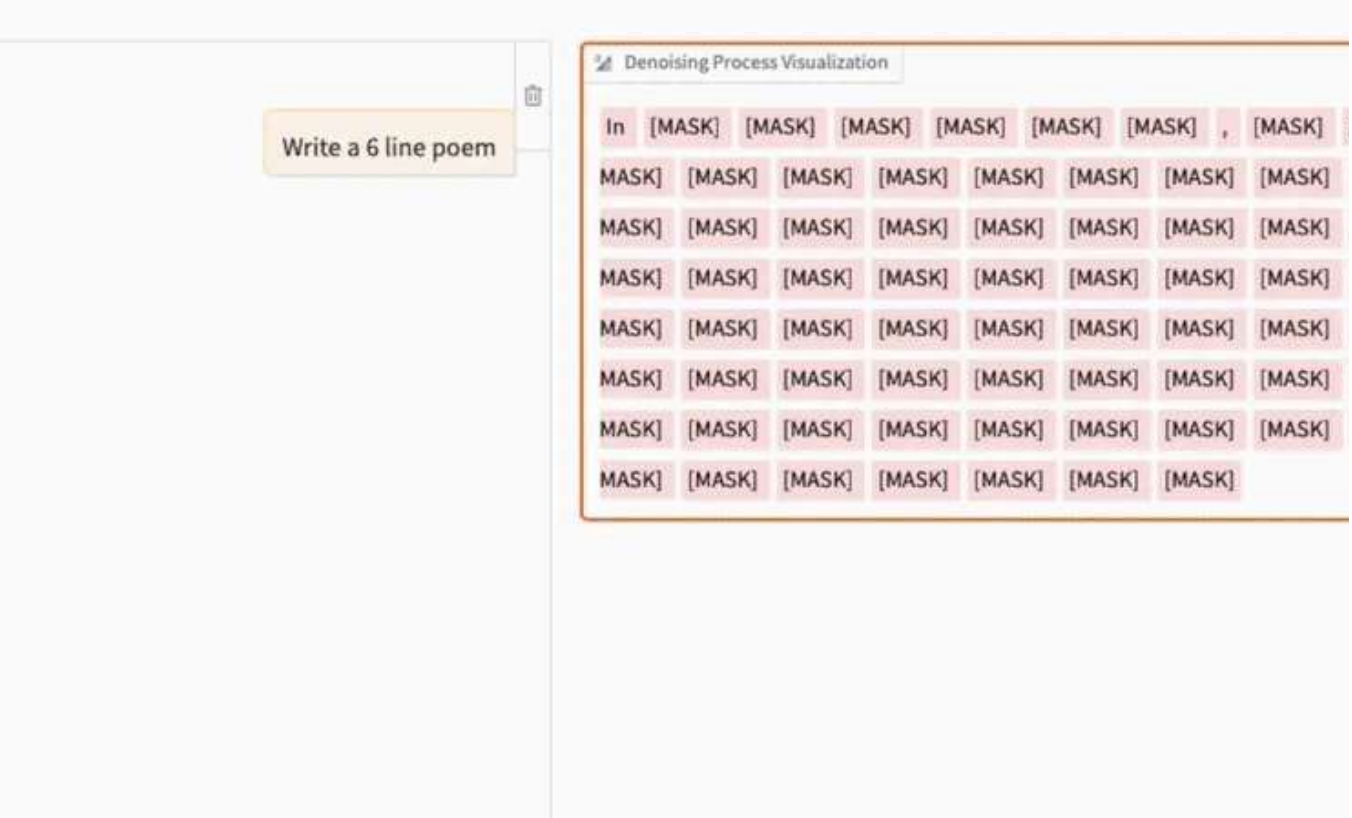


Figure 15: Illustration of the denoising process using the 8B LLaDA model.

As we can see in the animation above, the text diffusion process successively replaces [MASK] tokens with text tokens to generate the answer. If you are familiar with BERT ar masked language modeling, you can think of this diffusion process as an iterative appl of the BERT forward pass (where BERT is used with different masking rates).

Architecture-wise, diffusion LLMs are usually decoder-style transformers but without t causal attention mask. For instance, the aforementioned LLaDA model uses the Llama architecture. We call those architectures without a causal mask "bidirectional" as they l access to all sequence elements all at once. (Note that this is similar to the BERT architecture, which is called "encoder-style" for historical reasons.)

So, the main difference between autoregressive LLMs and diffusion LLMs (besides rer the causal mask) is the training objective. Diffusion LLMs like LLaDA use a generative diffusion objective instead of a next-token prediction objective.

In image models, the generative diffusion objective is intuitive because we have a cont pixel space. For instance, adding Gaussian noise and learning to denoise are mathema natural operations. Text, however, consists of discrete tokens, so we can't directly add c remove "noise" in the same continuous sense.

So, instead of perturbing pixel intensities, these diffusion LLMs corrupt text by progres masking tokens at random, where each token is replaced by a special mask token with specified probability. The model then learns a reverse process that predicts the missin tokens at each step, which effectively "denoises" (or unmasks) the sequence back to tl original text, as shown in the animation in Figure 15 earlier.

Explaining the math behind it would be better suited for a separate tutorial, but roughly can think about it as BERT extended into a probabilistic maximum-likelihood framewor

3.3 Autoregressive vs Diffusion LLMs

Earlier, I said that what makes diffusion LLMs appealing is that they generate (or denoi tokens in parallel instead of generating them sequentially as in a regular autoregressive This has the potential for making diffusion models more efficient than autoregressive L

That said, the autoregressive nature of traditional LLMs is one of their key strengths, th And the problem with pure parallel decoding can be illustrated with an excellent examp from the recent ParallelBench: Understanding the Trade-offs of Parallel Decoding in Di LLMs paper.

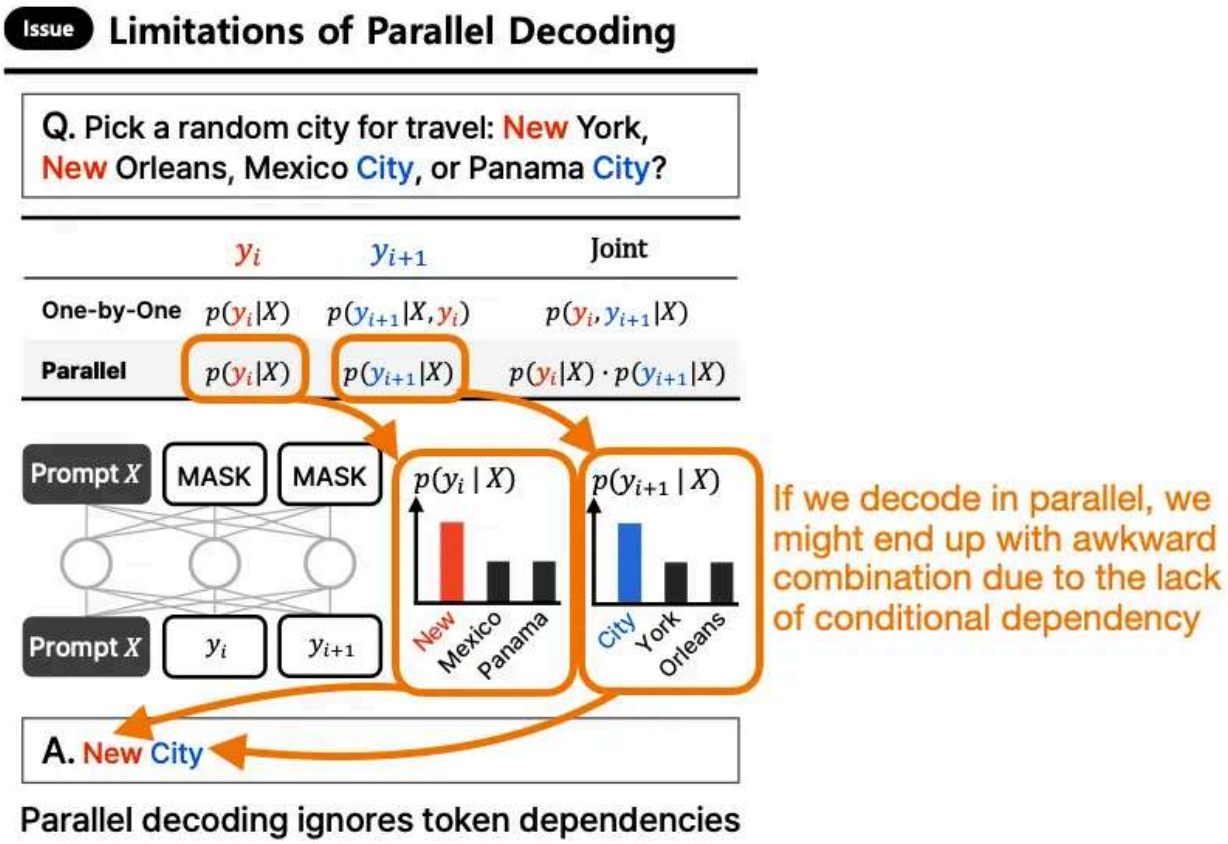


Figure 16: Annotated figure from ParallelBench: Understanding the Trade-offs of Parallel Decoding in Diffusion LLMs paper (https://arxiv.org/abs/2510.04767) showing the issue with parallel decoding.

For example, consider the following prompt:

> "Pick a random city for travel: New York, New Orleans, Mexico City, or Panama

> City?"

Suppose we ask the LLM to generate a two-token answer. It might first sample the tok "New" according to the conditional probability $p(y_t = \text{"New"} \mid X)$.

In the next iteration, it would then condition on the previously-generated token and like choose "York" or "Orleans," since both conditional probabilities

$p(y_{t+1} = \text{"York"} \mid X, y_t = \text{"New"})$ and $p(y_{t+1} = \text{"Orleans"} \mid X, y_t = \text{"New"})$

are relatively high (because "New" frequently co-occurs with these continuations in th training set). But if instead both tokens were sampled in parallel, the model might independently

select the two highest-probability tokens $p(y_t = \text{“New”} \mid X)$ and $p(y_{\{t+1\}} = \text{“City”} \mid X)$ lead awkward outputs like “New City.” (This is because the model lacks autoregressive conditioning and fails to capture token dependencies.)

In any case, the above is a simplification that makes it sound as if there is no conditional dependency in diffusion LLMs at all. This is not true. A diffusion LLM predicts all tokens parallel, as said earlier, but the predictions are jointly dependent through the iterative refinement (denoising) steps.

Here, each diffusion step conditions on the entire current noisy text. And tokens influence each other through cross-attention and self-attention in every step. So, even though all positions are updated simultaneously, the updates are conditioned on each other through shared attention layers.

However, as mentioned earlier, in theory, 20-60 diffusion steps may be cheaper than the 2000 inference steps in an autoregressive LLM when generating a 2000-token answer.

3.4 Text Diffusion Today

It’s an interesting trend that vision models adopt components from LLMs like attention and the transformer architecture itself, whereas text-based LLMs are getting inspired by pure vision models, implementing diffusion for text.

Personally, besides trying a few demos, I haven’t used many diffusion models yet, but I consider it a trade-off. If we use a low number of diffusion steps, we generate the answer faster but may produce an answer with degraded quality. If we increase the diffusion steps to generate better answers, we may end up with a model that has similar costs to an autoregressive one.

To quote the authors of the ParallelBench: Understanding the Trade-offs of Parallel Decoding in Diffusion LLMs paper:

> [...] we systematically analyse both [diffusion LLMs] and autoregressive LLMs, revealing that: (i) [diffusion LLMs] under parallel decoding can suffer dramatic quality degradation in real-world scenarios, and (ii) current parallel decoding strategies struggle to adapt

> degree of parallelism based on task difficulty, thus failing to achieve meaningful spe without compromising quality.

Additionally, another particular downside I see is that diffusion LLMs cannot use tools a of their chain because there is no chain. Maybe it's possible to interleave them betwee diffusion steps, but I assume this is not trivial. (Please correct me if I am wrong.)

In short, it appears that diffusion LLMs are an interesting direction to explore, but for nc they may not replace autoregressive LLMs. However, I can see them as interesting alternatives to smaller, on-device LLMs, or perhaps replacing smaller, distilled autoregr LLMs.

For instance, Google announced that it is working on a Gemini Diffusion model for text, they state

> Rapid response: Generates content significantly faster than even our fastest model

And while being faster, it appears that the benchmark performance remains on par with fast Gemini 2.0 Flash-Lite model. It will be interesting to see what the adoption and fee will be like once the model is released and users try it on different tasks and domains.

Category	Benchmark	Gemini Diffusion	Gemini 2.0 Flash-
Code	LiveCodeBench (v6)	30.9%	28
Code	BigCodeBench	45.4%	45
Code	LBPP (v2)	56.8%	56
Code	SWE-Bench Verified	22.9%	28
Code	HumanEval	89.6%	90
Code	MBPP	76.0%	75
Science	GPQA Diamond	40.4%	56
Mathematics	AIME 2025	23.3%	20
Reasoning	BIG-Bench Extra Hard	15.0%	21
Multilingual	Global MMLU (Lite)	69.1%	79

Figure 17: Benchmark performance of a (faster) diffusion LLM (Gemini Diffusion) versus a fast autoregressive LLM (Gemini 2.0 Flash-Lite). Based on the numbers reported in https://deepmind.google/models/gemini-diffusion/#capabilities.

4. World Models

So far, we discussed approaches that focused on improving efficiency and making mo faster or more scalable. And these approaches usually come at a slightly degraded mo performance.

Now, the topic in this section takes a different angle and focuses on improving modelin performance (not efficiency). This improved performance is achieved by teaching the r an "understanding of the world."

World models have traditionally been developed independently of language modeling, the recent Code World Models paper in September 2025 has made them directly relev this context for the first time.

Ideally, similar to the other topics of this article, world models are a whole dedicated art book) by themselves. However, before we get to the Code World Models (CWM) pape me provide at least a short introduction to world models.

4.1 The Main Idea Behind World Models

Originally, the idea behind world models is to model outcomes implicitly, i.e., to anticipa what might happen next without those outcomes actually occurring (as illustrated in th figure below). It is similar to how the human brain continuously predicts upcoming ever based on prior experience. For example, when we reach for a cup of coffee or tea, our b already predicts how heavy it will feel, and we adjust our grip before we even touch or li cup.

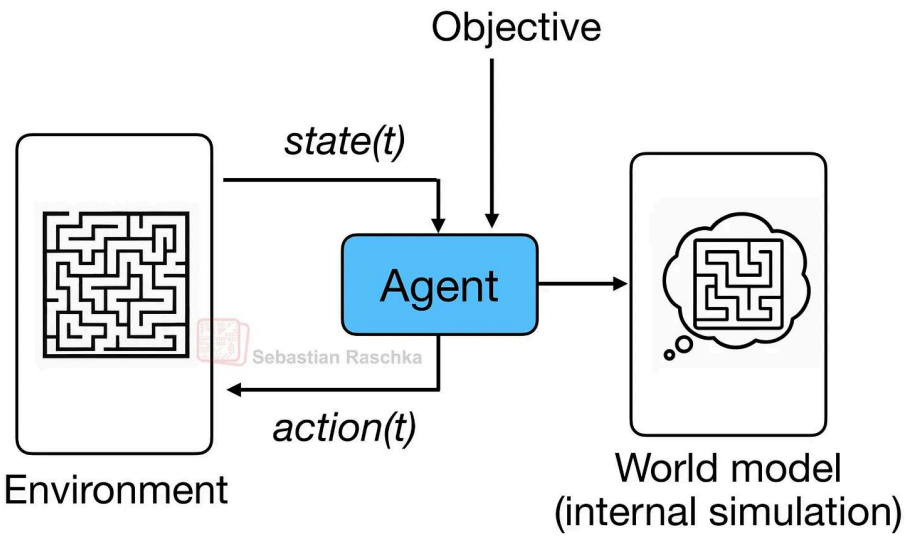


Figure 18: *Conceptual overview of a world model system. The agent interacts with the environment by observing its current state(t) and taking action(t) to achieve a given objective. In parallel, the agent learns an internal world model, which serves as a mental simulation of the environment, which allows it to predict outcomes and plan actions before executing them in the real world.*

The term "world model", as far as I know, was popularized by Ha and Schmidhuber's 20 paper of the same name: World Models, which used a VAE plus RNN architecture to lea internal environment simulator for reinforcement learning agents. (But the term or conc itself essentially just refers to modeling a concept of a world or environment, so it goes to reinforcement learning and robotics research in the 1980s.)

To be honest, I didn't have the new interpretation of world models on my radar until Yar LeCun's 2022 article A Path Towards Autonomous Machine Intelligence. It was essenti about mapping an alternative path to AI instead of LLMs.

4.2 From Vision to Code

That being said, world model papers were all focused on vision domains and spanned range of architectures: from early VAE- and RNN-based models to transformers, diffus models, and even Mamba-layer hybrids.

Now, as someone currently more focused on LLMs, the Code World Model paper (Sep 2025) is the first paper to capture my full attention (no pun intended). This is the first w model (to my knowledge) that maps from text to text (or, more precisely, from code to c

CWM is a 32-billion-parameter open-weight model with a 131k-token context windo Architecturally, it is still a dense decoder-only Transformer with sliding-window attentic Also, like other LLMs, it goes through pre-training, mid-training, supervised fine-tuning and reinforcement learning stages, but the mid-training data introduces the world-mod component.

4.3 Code World Models Vs Regular LLMs for Code

So, how does this differ from a regular code LLM such as Qwen3-Coder?

Regular models like Qwen3-Coder are trained purely with next-token prediction. They patterns of syntax and logic to produce plausible code completions, which gives them static text-level understanding of programming.

CWM, in contrast, learns to simulate what happens when the code runs. It is trained to predict the resulting program state, such as the value of a variable, after performing an like modifying a line of code, as shown in the figure below.

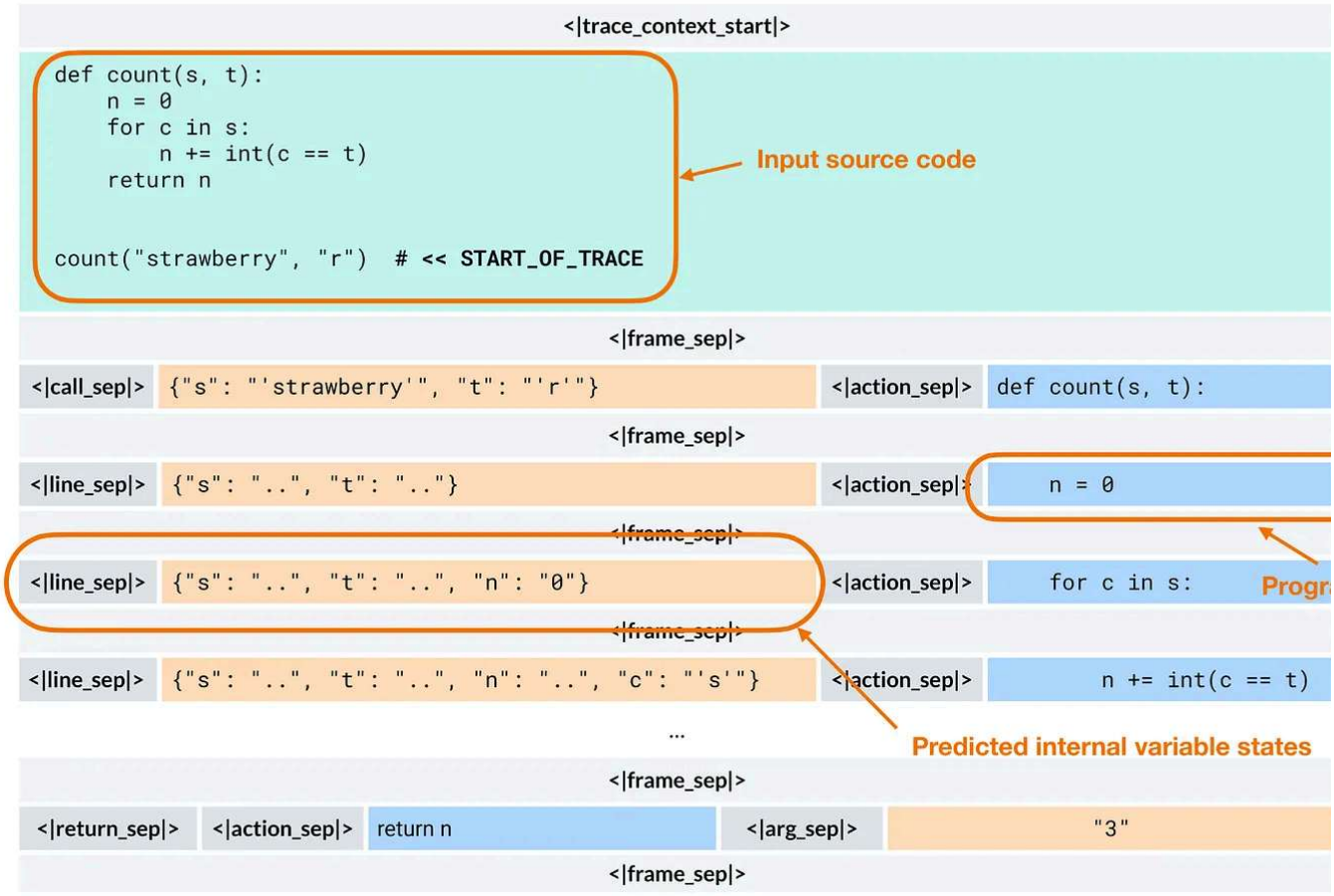


Figure 19: Example of code execution tracing in the Code World Model (CWM). The model predicts how variable states evolve step by step as each line of code executes. Here, the model effectively simulates the code's behavior. Annotated figure from https://www.arxiv.org/abs/2510.02387.

At inference time, CWM is still an autoregressive transformer that generates one token time, just like GPT-style models. The key difference is that these tokens can encode structured execution traces rather than plain text.

So, I would maybe not call it a world model, but a world model-augmented LLM.

For a first attempt, it performs surprisingly well, and is on par with gpt-oss-20b (mid reasoning effort) at roughly the same size.

If test-time-scaling is used, it even performs slightly better than gpt-oss-120b (high reasoning effort) while being 4x smaller.

Note that their test-time scaling uses a best@k procedure with generated unit tests (th a fancy majority voting scheme). It would have been interesting to see a tokens/sec or

to-solution comparison between CWM and gpt-oss, as they use different test-time-sc strategies (best@k versus more tokens per reasoning effort).

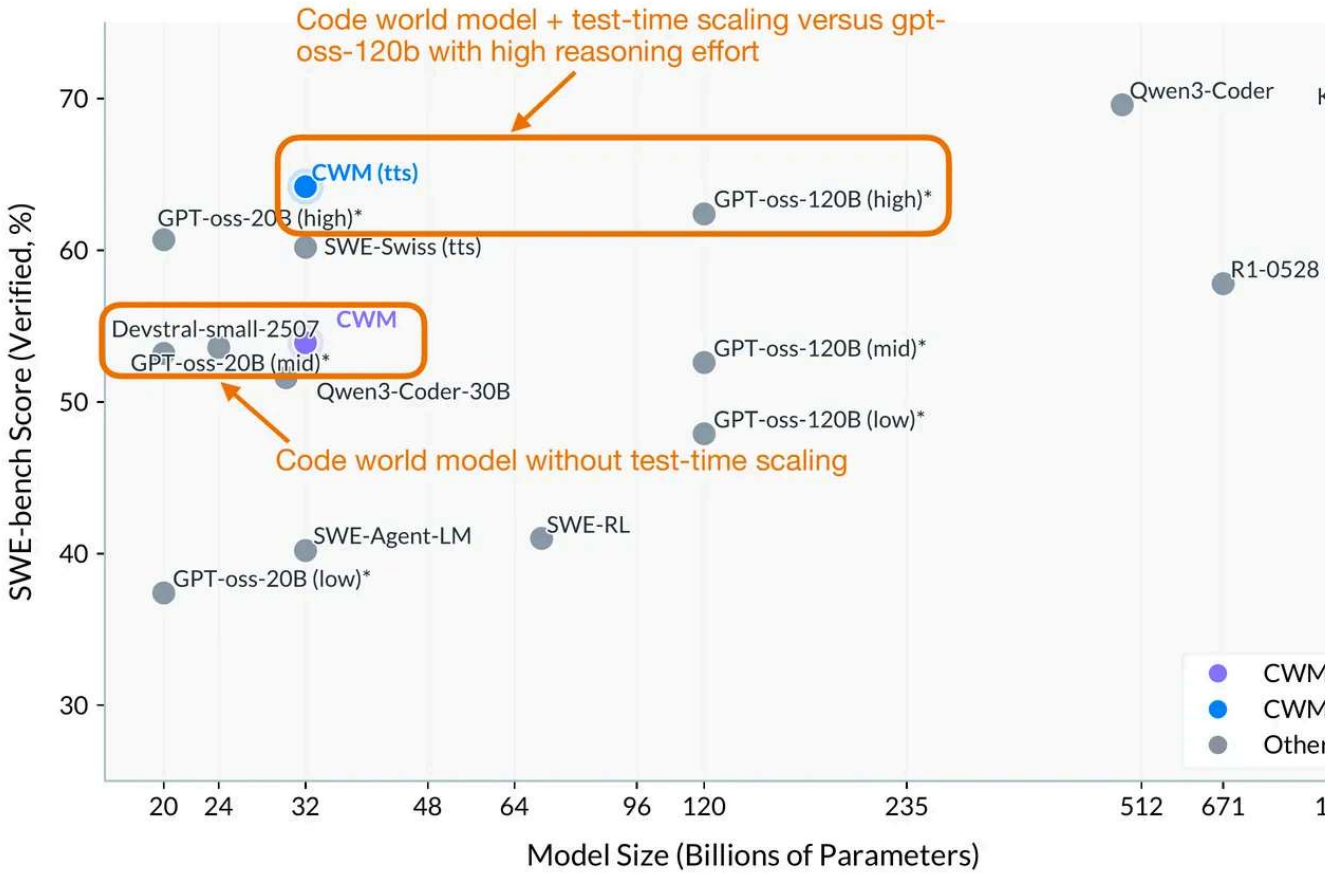


Figure 20: Performance of the code world model (CWM) compared to other popular LLMs on a coding benchmark (SWE-bench). Annotated figure from https://www.arxiv.org/abs/2510.02387.

5. Small Recursive Transformers

You may have noticed that all previous approaches still build on the transformer archite The topic of this last section does too, but in contrast to the models we discussed earli these are small, specialized transformers designed for reasoning.

Yes, reasoning-focused architectures don't always have to be large. In fact, with the Hierarchical Reasoning Model (HRM) a new approach to small recursive transformers l recently gained a lot of attention in the research community.

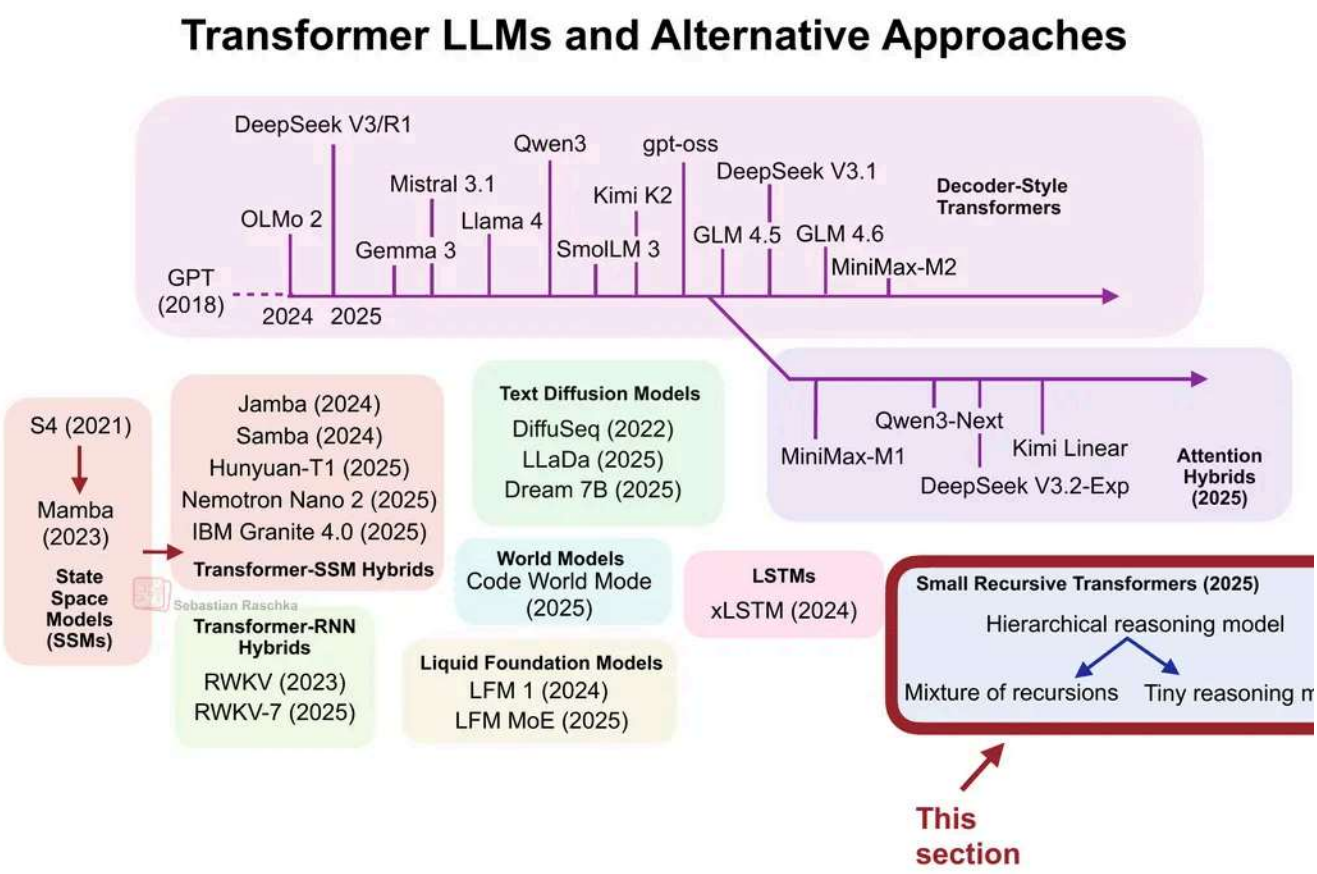


Figure 21: LLM landscape overview; this section small recursive transformers.

More specifically, the HRM developers showed that even very small transformer mode only 4 blocks) can develop impressive reasoning capabilities (on specialized problems when trained to refine their answers step by step. This resulted in a top spot on the AR(challenge.

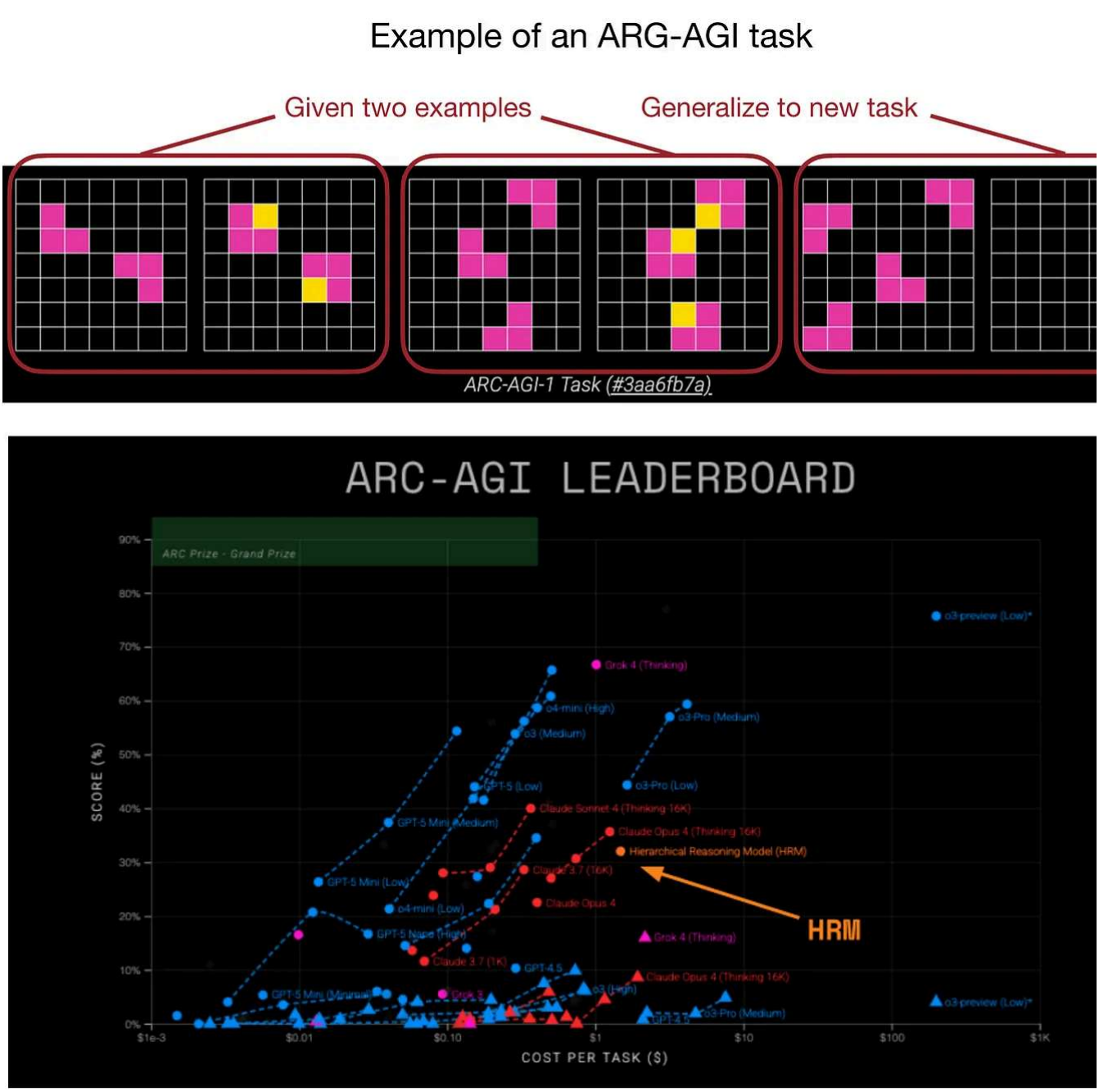


Figure 22: Example ARC-AGI 1 task (top) from arcprize.org/arc-agi/1 and the Hierarchical Reasoning Model (HRM) ranked on the leaderboard (bottom) from arcprize.org/blog/hrm-analysis.

The idea behind recursive models like HRM is that instead of producing an answer in o forward pass, the model repeatedly refines its own output in a recursive fashion. (As pa this process, each iteration refines a latent representation, which the authors see as the model's "thought" or "reasoning" process.)

The first major example was HRM earlier in the summer, followed by the Mixture-of-Recursions (MoR) paper.

And most recently, Less is More: Recursive Reasoning with Tiny Networks (October 20 proposes the Tiny Recursive Model (TRM, illustrated in the figure below), which is a sin and even smaller model (7 million parameters, about 4× smaller than HRM) that perfor even better on the ARC benchmark.

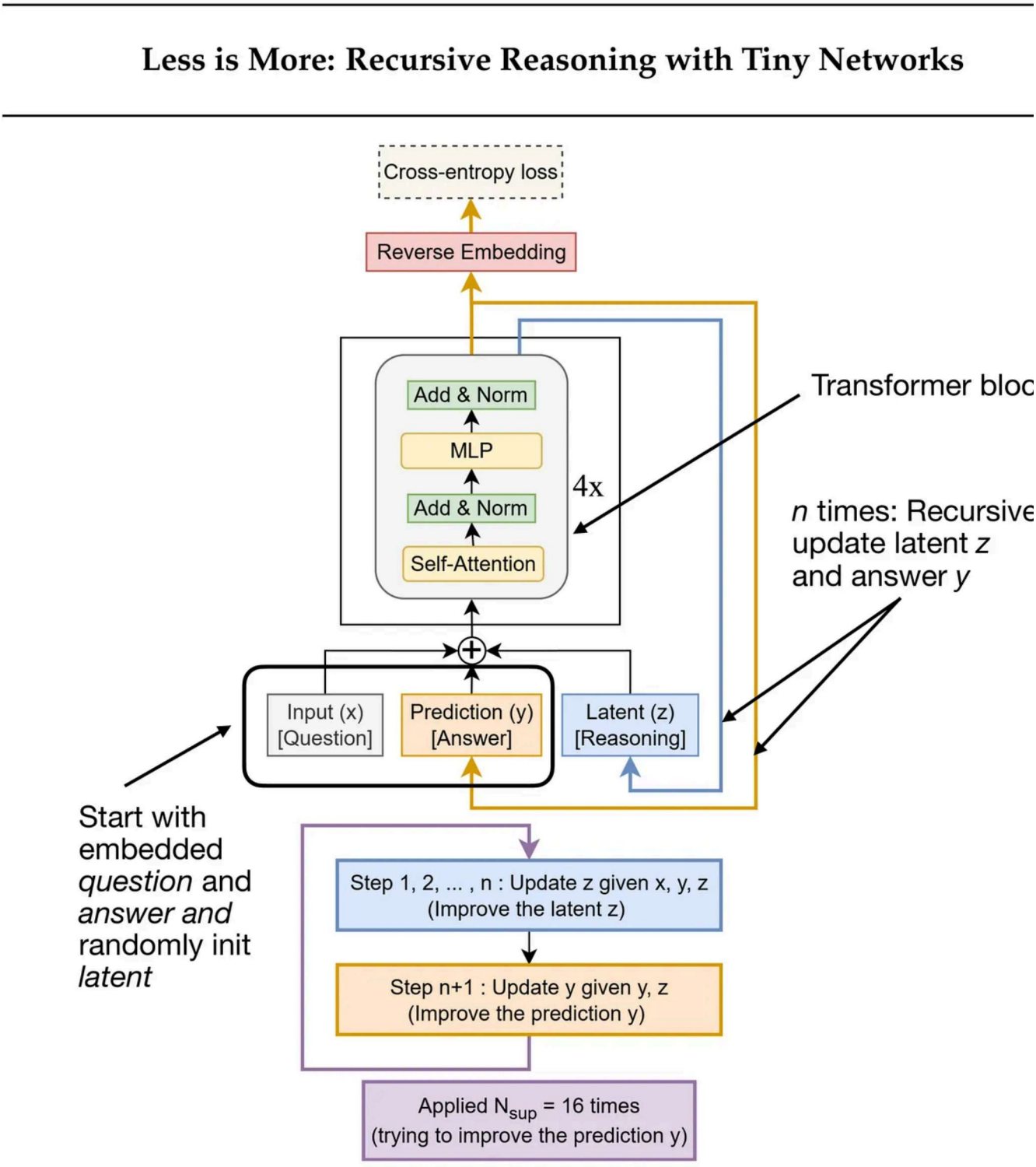


Figure 23: The Tiny Recursive Model (TRM). Annotated figure from https://arxiv.org/abs/2510.04871.

In the remainder of this section, let's take a look at TRM in a bit more detail.

5.1 What Does Recursion Mean Here?

TRM refines its answer through two alternating updates:

1. It computes a latent reasoning state from the current question and answer.
2. It then updates the answer based on that latent state.

The training runs for up to 16 refinement steps per batch. Each step performs several n grad loops to iteratively refine the answer. This is followed by a gradient loop that backpropagates through the full reasoning sequence to update the model weights.

It's important to note that TRM is not a language model operating on text. However, bec (a) it's a transformer-based architecture, (b) reasoning is now a central focus in LLM re and this model represents a distinctly different take on reasoning, and (c) many reader asked me to cover HRM (and TRM is its more advanced successor) I decided to includ here.

While TRM could be extended to textual question-answer tasks in the future, TRM curr works on grid-based inputs and outputs. In other words, both the "question" and the "answer" are grids of discrete tokens (for example, 9×9 Sudoku or 30×30 ARC/Maze puzzles), not text sequences.

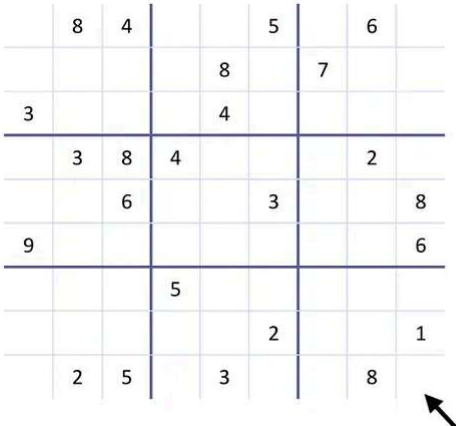
5.2 How Does TRM Differ From HRM?

HRM consists of two small transformer modules (each 4 blocks) that communicate ac recursion levels. TRM only uses a single 2-layer transformer. (Note that the previous TF figure shows a 4× next to the transformer block, but that's likely to make it easier to cor against HRM.)

TRM backpropagates through all recursive steps, whereas HRM only backpropagates through the final few.

HRM includes an explicit halting mechanism to determine when to stop iterating. TRM replaces this mechanism with a simple binary cross-entropy loss that learns when to s iterating.

Performance-wise, TRM performs really well compared to HRM, as shown in the figure



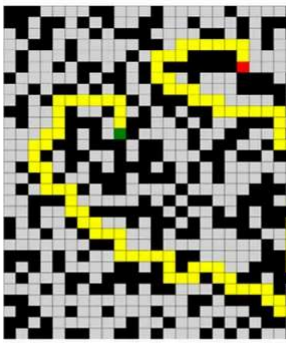


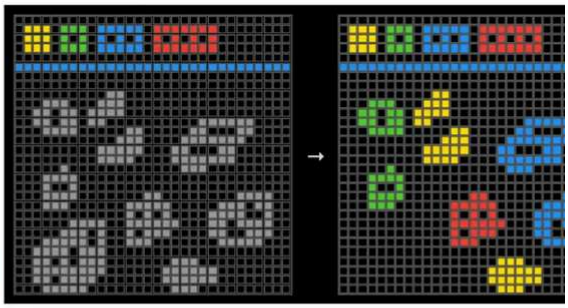
Table 4. % Test accuracy on Puzzle Benchmarks (Sudoku-Extreme and Maze-Hard)

Method	# Params	Sudoku	Maze
Chain-of-thought, pretrained			
Deepseek R1	671B	0.0	0.0
Claude 3.7 8K	?	0.0	0.0
O3-mini-high	?	0.0	0.0
Direct prediction, small-sample training			
Direct pred	27M	0.0	0.0
HRM	27M	55.0	74.5
TRM-Att (Ours)	7M	74.7	85.3
TRM-MLP (Ours)	5M/19M[1]	87.4	0.0

TRM-MLP is a variant that replaces the self-attention mechanism by a more expensive MLP layer

Table 5. % Test accuracy on ARC-AGI Benchmarks (2 tries)

Method	# Params	ARC-1	ARC-2
Chain-of-thought, pretrained			
Deepseek R1	671B	15.8	1.3
Claude 3.7 16K	?	28.6	0.7
o3-mini-high	?	34.5	3.0
Gemini 2.5 Pro 32K	?	37.0	4.9
Grok-4-thinking	1.7T	66.7	16.0
Bespoke (Grok-4)	1.7T	79.6	29.4
Direct prediction, small-sample training			
Direct pred	27M	21.0	0.0
HRM	27M	40.3	5.0
TRM-Att (Ours)	7M	44.6	7.8
TRM-MLP (Ours)	19M	29.6	2.4



TRM outperforms HM on both ARC-1 and the more challenging ARC-2

Figure 24: Performance comparison of the Hierarchical Reasoning Model (HRM) and Recursive Model (TRM).

The paper included a surprising number of ablation studies, which yielded some interesting additional insights. Here are two that stood out to me:

1. Fewer layers leads to better generalization. Reducing from 4 to 2 layers improved Sudoku accuracy from 79.5% to 87.4%.
2. Attention is not required. Replacing self-attention with a pure MLP layer also improved accuracy (74.7% to 87.4%). But this is only feasible here because the context is small and fixed-length.

5.3 The Bigger Picture

While HRM and TRM achieve really good reasoning performance on these benchmarks, comparing them to large LLMs is not quite fair. HRM and TRM are specialized models for tasks like ARC, Sudoku, and Maze pathfinding, whereas LLMs are generalists. Sure, HRM and TRM can be adopted for other tasks as well, but they have to be specially trained on each task. So, in that sense, we can perhaps think of HRM and TRM as efficient pocket calculators, whereas LLM are more like computers, which can do a lot of other things as well.

Still, these recursive architectures are exciting proof-of-concepts that highlight how small, efficient models can "reason" through iterative self-refinement. Perhaps, in the future, such models could act as reasoning or planning modules embedded within larger tool-using systems.

For now, LLMs remain ideal for broad tasks, but domain-specific recursive models like these can be developed to solve certain problems more efficiently once the target domain is well understood. Beyond the Sudoku, Maze finding, and ARC proof-of-concept benchmarks, there are possibly lots of use cases in the physics and biology domain where such models could find use.

As an interesting tidbit, the author shared that it took less than $500 to train this model on 4 H100s for around 2 days. I am delighted to see that it's still possible to do interesting

without a data center.

6. Conclusion

I originally planned to cover all models categories in the overview figure, but since the a ended up longer than I expected, I will have to save xLSTMs, Liquid Foundation Models Transformer-RNN hybrids, and State Space Models for another time (although, Gated DeltaNet already gave a taste of State Space Models and recurrent designs.)

As a conclusion to this article, I want to repeat the earlier words, i.e., that standard autoregressive transformer LLMs are proven and have stood the test of time so far. The also, if efficiency is not the main factor, the best we have for now.

Traditional Decoder-Style, Autoregressive Transformers

+ Proven & mature tooling
+ “well-understood”
+ Scaling laws
+ SOTA
- Expensive training
- Expensive inference (except for aforementioned tricks)

If I were to start a new LLM-based project today, autoregressive transformer-based LL would be my first choice.

I definitely find the upcoming attention hybrids very promising, which are especially interesting when working with longer contexts where efficiency is a main concern.

Linear Attention Hybrids

+ Same as decoder-style transformers
+ Cuts FLOPs/KV memory at long-context tasks
- Added complexity
- Trades a bit of accuracy for efficiency

On the more extreme end, text diffusion models are an interesting development. I'm stil somewhat skeptical about how well they perform in everyday use, as I've only tried a fe quick demos. Hopefully, we'll soon see a large-scale production deployment with Goog Gemini Diffusion that we can test on daily and coding tasks, and then find out how pec actually feel about them.

Text Diffusion Models

+ Iterative denoising is a fresh idea for text
+ Better parallelism (no next-token dependence)
- Can't stream answers
- Doesn't benefit from CoT?
- Tricky tool-calling?
- Solid models but not SOTA

While the main selling point of text diffusion models is improved efficiency, code world models sit on the other end of the spectrum, where they aim to improve modeling performance. As of this writing, coding models, based on standard LLMs, are mostly improved through reasoning techniques, yet if you have tried them on trickier challenge have probably noticed that they (more or less) still fall short and can't solve many of th trickier coding problems well.

I find code world models particularly interesting and believe they could be an importan step toward developing more capable coding systems.

Code World Model

+ Promising approach to improve code understanding
+ Verifiable intermediate states
- Inclusion of executable code traces complicates training
- Code running adds latency

Lastly, we covered small recursive transformers such as hierarchical and tiny reasoning models. These are super interesting proof-of-concept models. However, as of today, th primarily puzzle solvers, not general text or coding models. So, they are not in the same

category as the other non-standard LLM alternatives covered in this article. Nonethele they are very interesting proofs-of-concept, and I am glad researchers are working on

Right now, LLMs like GPT-5, DeepSeek R1, Kimi K2, and so forth are developed as spec purpose models for free-form text, code, math problems and much more. They feel like force and jack-of-all-trades approach that we use on a variety of tasks, from general knowledge questions to math and code.

However, when we perform the same task repeatedly, such brute-force approaches be inefficient and may not even be ideal in terms of specialization. This is where tiny recur transformers become interesting: they could serve as lightweight, task-specific model are both efficient and purpose-built for repeated or structured reasoning tasks.

Also, I can see them as potential "tools" for other tool-calling LLMs; for instance, when use Python or calculator APIs to solve math problems, special tiny reasoning models c this niche for other types of puzzle- or reasoning-like problems.

Small Recursive Transformers

+ Very small architecture
+ Good generalization on puzzles
- Special purpose models
- Limited to puzzles (so far)

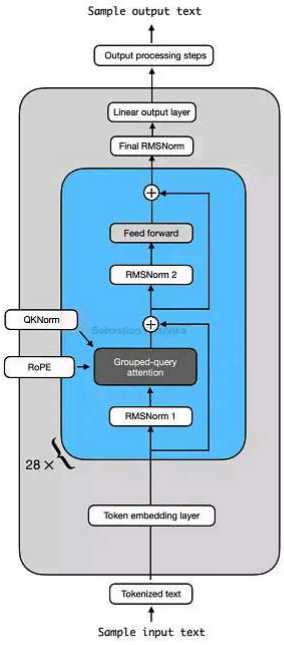
This has been a long article, but I hope you discovered some of the fascinating approa that often stay outside the spotlight of mainstream LLMs.

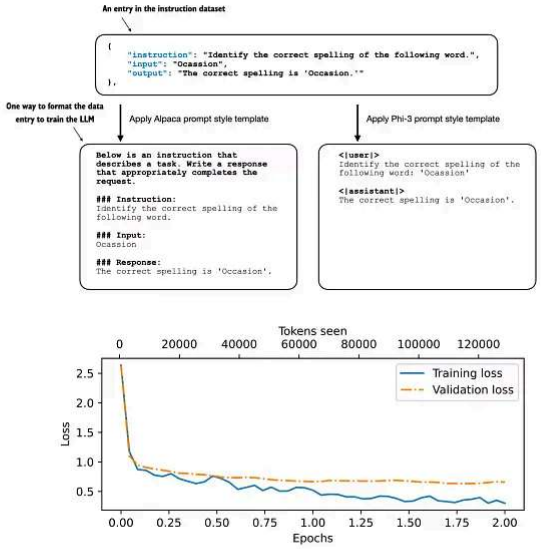
And if you've been feeling a bit bored by the more or less conventional LLM releases, I this helped rekindle your excitement about AI again because there's a lot of interesting happening right now!

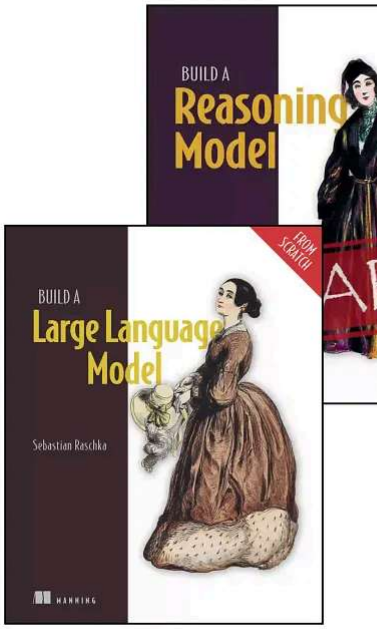
This magazine is a personal passion project, and your support helps keep it alive.

If you'd like to support my work, please consider my Build a Large Language Model (Scratch) book or its follow-up, Build a Reasoning Model (From Scratch). (I'm confide you'll get a lot out of these; they explain how LLMs work in depth you won't find elsewhere.)

Thanks for reading, and for helping support independent research!







Build a Large Language Model (From Scratch) is now available on Amazon. Build a Reasoning Model (From Scratch) is in Early Access at Manning.

If you read the book and have a few minutes to spare, I'd really appreciate a brief review helps us authors a lot!

Your support means a great deal! Thank you!

384 Likes · 37 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Kai Liu 2025年11月4日

Liked by Sebastian Raschka, PhD

Great article, Sebastian! Thank you for your work!

LIKE (3) REPLY

siyu siyu 2月10日

Liked by Sebastian Raschka, PhD

Looks like you have the same (similar) code for both "Gated Attention" vs. "Gated delt attention". At least the subroutines have the same name, and I'm having trouble seein difference between the 2 implementations

LIKE (1) REPLY

1 reply by Sebastian Raschka, PhD

26 more comments...